

uc3m | Universidad **Carlos III** de Madrid

Master Degree in Cybersecurity

Academic Year: 2024-2025

Master Thesis “The Game Beneath the Game:
The Evolution of Cheats and Anti-Cheat
Systems in Counter-Strike 2”

Pablo Valiente

Tutor: Antonio Nappa
Universidad Carlos III de Madrid
Date: June 2025

Abstract

This thesis examines the evolution of cheating in video games, from early debugging tools to modern external and internal exploits. It explores key cheating techniques, anti-cheat countermeasures, and the ongoing arms race between both sides. Additionally, a practical implementation of a cheat in **Counter-Strike 2** is developed to assess the effectiveness of contemporary anti-cheat systems and identify potential vulnerabilities.

Keywords: Cheating, Video Games, Counter-Strike 2, Anti-Cheat Systems, Game Hacking, Memory Manipulation

DEDICATION

CONTENTS

1	Introduction	2
2	From Primitive Survival to Digital Domination	6
1	The Competitive Nature of Humanity	6
2	The Birth of Computation and the Internet: A New Digital Battleground	7
2.1	Early Computational Tools: From the Abacus to Mechanical Calculators	7
2.2	The 19th-Century Revolution: The Birth of Programmable Machines	7
2.3	The Acceleration of Computation in the 20th Century	8
2.3.1	World War II and the Role of Computation	8
2.3.2	Post-War Expansion: From Military to Commercial Computing	8
2.4	The Birth of the Internet: From Military Networks to Global Infrastructure	9
2.5	The Digital Arena: From Computing to Competition	9
2.6	From Networks to Virtual Battlefields	10
3	The Evolution of Cheating: From Debugging Tools to an Organized Industry	11
1	The Early Days: Debugging Tools and Experimental Modifications	11
2	The Rise of Cheating in Consoles and PC	11
2.1	Cheat Devices: Game Genie, GameShark, and Pro Action Replay	12
3	The Transition to Online Gaming: Cheating Becomes a Competitive Threat	13
4	The Birth and Evolution of Anti-Cheat Systems	13
5	The Commercialization of Cheating	16
4	The Prevalence and Impact of Cheating in Online Multiplayer Games	19
1	Cheating as a Critical Threat to Player Retention	19
2	Case Study: Cheating in <i>Counter-Strike 2</i>	20
2.1	Trust Factor and Reporting System Limitations	21
2.2	Economic and Psychological Impact	21
3	Community Perception and the Culture of Cynicism	22
4	Legal and Ethical Responses to Cheat Development	22
5	Anti-Cheat Technologies: Current Trends and Limitations	22
6	Toward a More Resilient Competitive Ecosystem	23

5	Injection and Detection Techniques	24
1	Why Injection Matters in Cheat Development	24
2	Overview of Code Injection Techniques	24
3	Detection Techniques in Anti-Cheat Systems	26
4	Evasion Techniques Used by Cheats	28
5	Extended VAC Capabilities and Developer Countermeasures	30
6	Function Hooking and Trampoline-Based Interception	31
1	What is a Trampoline Hook?	31
2	Reverse Engineering and Function Discovery	32
3	Why Trampoline Hooks Are Effective	33
4	Implementing Hooks with MinHook	33
5	Final Thoughts on Trampoline Hooking	35
7	Internal Game Structure: Logic Behind Counter-Strike 2	36
1	Understanding Entity Lists in Video Games	36
1.1	Common Structures for Entity Lists	36
2	Entity Organization in Counter-Strike 2	37
3	Entity List Traversal	39
3.1	Locating and Identifying the Entity List in Memory	39
3.2	Reverse Engineering the Engine's Entity Resolution Logic	42
3.2.1	Efficient Entity Tracking: Hooking OnAddEntity and OnRemoveEntity	43
8	Cheat Features Implemented in Our Cheat for CS2	44
1	Chams: Enhancing Visibility Through Engine-Based Rendering	45
1.1	Extracting Default Materials from the Game Engine	46
1.2	Generating Custom Materials	47
1.3	Hooking CreateMaterial and Injecting Custom Materials	49
1.4	Loading KV3 Materials into the Game	49
1.5	Hooking OnDrawModelExecute and Overriding Materials	51
1.5.1	Filtering Target Entities	51
1.5.2	Overriding Material Pointers	52
1.5.3	Visible vs Non-Visible Chams Logic	52
2	Aimbot: Automated Aim to Enemy Players	54
2.1	Entity Management: Hooking the Game's Entity System	54
2.1.1	Computing Aiming Angles	56
2.1.2	Deriving the Direction Vector	56
2.1.3	Computing the Player's Position	57
2.1.4	Centering Coordinates on the Enemy	57
2.1.5	Local Coordinate System and Angle Calculation	58
2.1.6	Mathematical Computation of Aiming Angles	58
2.2	Overwriting Player View Angles	60

2.3	Smoothing for Stealth	60
3	Bone Calculation: Extracting and Rendering Player Skeletons	61
3.1	Extracting Bone Data from Player Entities	61
3.2	Visualizing Bone Indices on the Player Model	62
3.3	Rendering a Connected Skeleton	63
4	Anti-Aim: Misleading Enemy Targeting Systems	64
4.1	What Is Anti-Aim?	64
4.2	Types of Anti-Aim	65
4.3	Lower Body Yaw (LBY) and the LBY Breaker	65
4.4	Benefits for the Cheater	66
4.5	Importance in Modern Cheats	66
	Project Timeline (Gantt Chart)	68
	Bibliography	71

LIST OF FIGURES

2.1	LAN party in the late 1990s—a grassroots precursor to modern esports. .	10
3.1	Game Genie	12
3.2	Chronological overview of key anti-cheat systems introduced between 2000 and 2023.	15
4.1	Evolution of Average CS2 Player Counts (Jan 2024 – Apr 2025). Source: SteamCharts.	20
5.1	Simplified flowchart of dynamic module injection using Manual Mapping.	25
5.2	Example of VMT Hooking.	27
6.1	Control flow in a trampoline hook setup.	32
8.1	Created Chams Materials	52
8.2	Enter Caption	53
8.3	Representation of Pitch, Yaw, and Roll in a 3D Space.	56
8.4	Representation of the player and enemy positions on the map.	57
8.5	Vista original con los ejes coordenados.	58
8.6	Cálculo del vector de desplazamiento D	58
8.7	3D representation of the aiming direction vector.	59
8.8	Visualizing all bone indices rendered over the player model.	62
8.9	Simplified player skeleton rendered using bone connections.	64

LIST OF TABLES

- 3.1 Overview of major CS2 cheat providers, including release year, pricing models, and access restrictions. 16
- 4.1 Survey Results on Cheating in Online Games (Irdeto, 2018) 19
- 4.2 Activity of Banned Accounts Prior to Enforcement (Jan–Apr 2024) 21
- 8.1 Core Functionalities Overview 44

1. INTRODUCTION

Context and Motivation

The transformation of video games from a niche hobby to a dominant force in the entertainment industry has fundamentally reshaped how individuals interact, compete, and even build careers. In this new era of **competitive gaming**, players engage in high-stakes environments that reward precision, strategy, and reflexes. Fueled by global connectivity and high-speed internet, online multiplayer games have evolved into structured **competitive ecosystems**—complete with rankings, seasonal events, and financial incentives—that mirror traditional sports in both scope and intensity.

Video games now serve not only as entertainment, but also as **social platforms, economic drivers, and professional arenas**. Yet, this rapid evolution has also given rise to a persistent and multifaceted threat: **cheating**. By leveraging unauthorized software modifications, players seek unfair advantages that distort the competitive balance. These exploits range from memory manipulation and external assistance tools to hardware-based hacks and network-level abuses. The resulting **arms race** between developers—who continually enhance anti-cheat mechanisms—and cheat creators—who innovate to bypass them—has become one of the most dynamic and technically challenging conflicts in the digital age.

Furthermore, the deployment of modern anti-cheat systems introduces complex **privacy, legal, and environmental concerns**. Effective cheat detection often requires invasive system-level access, raising red flags regarding user privacy and digital rights. Simultaneously, the commercialization of cheating software generates substantial financial losses for developers and presents regulatory challenges on a global scale. Additionally, the increased computational demand of sophisticated detection systems contributes to a growing environmental footprint.

To move beyond theoretical discourse, this thesis adopts a hands-on approach: the implementation and analysis of a custom-built cheat in a large-scale multiplayer game. This methodology enables a detailed examination of anti-cheat technologies from a practical perspective, offering empirical insights into their design, effectiveness, and limitations.

By combining historical context, technical exploration, and real-world experimentation, this research aims to contribute a comprehensive understanding of cheating in competitive gaming—its motivations, its mechanisms, and its broader societal implications.

Objectives and Scope

This study seeks to investigate cheating in competitive video games through both theoretical analysis and practical experimentation. The research is structured around three main objectives:

1. **To examine the historical evolution of cheating in gaming**, analyzing its impact on game design, player behavior, and industry responses.
2. **To provide a technical breakdown of common cheating techniques**, such as aimbots, wallhacks, memory injection, and network manipulation, along with their consequences for competitive fairness.
3. **To implement and evaluate a custom cheat in a contemporary multiplayer game**, assessing the robustness of current anti-cheat technologies through empirical testing and reverse engineering.

Competencies and Alignment with the Master's Program

This thesis is closely aligned with the core competencies of the **Master's Degree in Cybersecurity**, particularly in areas such as **reverse engineering**, **intrusion analysis**, and **digital forensics**. Through this work, the following competencies will be developed and demonstrated:

Basic Competencies

- **[CB6]** Conduct original research into the security challenges of competitive gaming, with a focus on cheating and countermeasures.
- **[CB7]** Apply cybersecurity methodologies to the domain of game security, including the development, detection, and evasion of cheats.
- **[CB8]** Critically assess the legal and ethical implications of game manipulation and unauthorized software modifications.

General Competencies

- **[CG1]** Investigate and replicate real-world intrusion techniques to understand vulnerabilities in game environments.
- **[CG2]** Analyze the architecture and operation of anti-cheat systems in order to evaluate their efficacy and limitations.
- **[CG3]** Produce high-quality technical documentation covering offensive security techniques and their forensic analysis.

Specific Competencies

- [CE1] Understand the psychological and social drivers behind cheating in competitive settings.
- [CE2] Explore advanced cheat evasion strategies and their implications for detection.
- [CE3] Track trends in game security threats and relate them to broader challenges in cybersecurity.
- [CE4] Perform forensic examinations of game processes and memory to detect unauthorized interventions.
- [CE6] Assess the internal workings and performance trade-offs of modern anti-cheat technologies.

Document Structure

This thesis is organized as follows:

- **Chapter 1: Introduction** – Outlines the motivation, objectives, competencies, and structure of the study.
- **Chapter 2: From Primitive Survival to Digital Domination** – Investigates the human instinct for competition and its evolution into virtual arenas, tracing the technological milestones that gave rise to competitive gaming.
- **Chapter 3: The Evolution of Cheating: From Debugging Tools to an Organized Industry** – Explores the origins of game cheating, its progression into an underground economy, and the industry's adaptive response via anti-cheat systems.
- **Chapter 4: The Prevalence and Impact of Cheating in Online Multiplayer Games** – Analyzes the consequences of cheating on game balance, player trust, and industry economics, with a focus on Counter-Strike 2 as a case study.
- **Chapter 5: Injection and Detection Techniques** – Provides a technical overview of code injection methods, detection strategies, and the ongoing battle between cheaters and anti-cheat developers.
- **Chapter 6: Internal Game Structure: Logic Behind Counter-Strike 2** – Dissects the architecture of the CS2 engine, including memory layouts, entity management, and reverse engineering methods.
- **Chapter 7: Cheat Features Implemented in Our Cheat for CS2** – Presents the design and implementation of various cheat modules, including visibility enhancements, automated aiming, and anti-detection features.

2. FROM PRIMITIVE SURVIVAL TO DIGITAL DOMINATION

1 The Competitive Nature of Humanity

Humanity's relentless pursuit of **superiority, recognition, and dominance** has been a constant throughout history. From early survival strategies to the era of modern technology, the drive to **outperform others** has been a decisive factor in advances across science, warfare, politics—and, more recently, digital realms.

This competitive impulse goes beyond simple ambition; it is a **deeply ingrained psychological mechanism** that has shaped human behavior for centuries. As Festinger [1] explained through his **Social Comparison Theory**, individuals are naturally inclined to evaluate themselves against their peers, a process that reinforces competition by seeking validation and status. In a similar vein, McClelland [2] introduced the **Achievement Motivation Theory** to demonstrate that the inherent desire to succeed and surpass others is crucial not only for personal fulfillment but also for social standing.

Viewed from an **evolutionary perspective**, competition has been essential for survival. Early human societies relied on traits such as strategic thinking, adaptability, and cooperative behaviors to secure resources, defend territories, and outcompete rival groups. Over time, these traits not only ensured survival but also influenced the development of complex **cultural, intellectual, and economic systems**. Even from a young age, individuals instinctively compare themselves with others, perpetuating a competitive framework embedded in our cognition and social interactions.

What began as a survival mechanism has evolved into a multifaceted force that continues to shape human ambition and societal progress.

The advent of modern technology has transferred this age-old drive into **digital environments**. The development of **computers** and the emergence of the **internet** revolutionized human interaction, information processing, and competitive engagement. In particular, video games have emerged as a potent platform that encapsulates competitive spirit through structured, goal-oriented systems—offering challenges against both artificial intelligence and human opponents.

These digital platforms mirror the essence of traditional competition by demanding skill optimization, the discovery of hidden mechanics, and the exploitation of strategic advantages. Empirical studies reveal that under competitive pressure, individuals are prone to

seek alternative methods to gain an edge, especially when conventional avenues for success are obstructed [3]. In the realm of online multiplayer gaming, the presence of **ranking systems, leaderboards, and social validation** further intensifies this competitive drive, urging players to pursue victory through mastery, strategy, or—even through exploitation.

To grasp how competition has been reinvented in the digital age, it is necessary to explore the historical evolution that laid the groundwork for these advancements. The subsequent sections will trace the development of **computation**, the advent of **networked systems**, and the rise of **video games** as dominant cultural and competitive forces. Through this historical lens, we reveal how ancient competitive instincts have been transposed into virtual battlegrounds where **skill, innovation, and adaptability** are the keys to success.

2 The Birth of Computation and the Internet: A New Digital Battleground

The roots of digital competition lie in humanity’s enduring efforts to **automate reasoning and computation**. From rudimentary counting devices to complex mechanical systems, civilizations have continually sought to enhance their ability to solve mathematical problems—an essential skill for the advancement of commerce, astronomy, and engineering.

2.1 Early Computational Tools: From the Abacus to Mechanical Calculators

To improve accuracy and efficiency in arithmetic operations, ancient societies devised tools that supported numerical reasoning. Among the earliest was the **abacus** (circa 2400 BCE), which saw widespread use in Mesopotamia, China, and Europe. This manually operated device enabled rapid calculations and laid the groundwork for future developments in computational thinking.

By the 17th century, the integration of mechanical engineering into mathematical problem-solving gave rise to more sophisticated inventions. One notable example is **Pascal’s Adding Machine** (1642), designed by **Blaise Pascal**. This mechanical calculator employed interlocking gears to automate basic arithmetic, significantly reducing the cognitive load of manual computation and heralding a shift toward mechanized processing.

2.2 The 19th-Century Revolution: The Birth of Programmable Machines

While earlier devices focused on performing fixed calculations, the 19th century introduced a paradigm shift with the conceptualization of **programmable machines**. Cen-

tral to this revolution was **Charles Babbage**, whose design for the **Analytical Engine** introduced the foundational principles of modern computing, including conditional branching, memory storage, and modular input/output.

Expanding on Babbage's work, **Ada Lovelace** recognized that such machines could go beyond numerical tasks. Her algorithm for the Analytical Engine is widely regarded as the first computer program. More significantly, her insights into symbolic computation and algorithmic design anticipated the emergence of **general-purpose computing** and modern programming languages.

2.3 The Acceleration of Computation in the 20th Century

The 20th century brought exponential advances in computing, driven largely by scientific innovation and geopolitical demands. The transition from mechanical apparatuses to fully electronic systems revolutionized data processing, transforming computation into a critical enabler of technological and industrial growth.

2.3.1 World War II and the Role of Computation

The exigencies of **World War II** played a pivotal role in accelerating computational innovation. **Alan Turing's** formulation of the **Turing Machine** provided the theoretical framework for algorithmic processing, while his practical work in cryptanalysis at Bletchley Park—particularly in deciphering the **Enigma cipher**—demonstrated the immense strategic value of computational power.

Simultaneously, pioneering machines such as **Colossus** (UK, 1943) and **ENIAC** (US, 1945) were developed to perform large-scale calculations for military applications, including cryptography and ballistic modeling. These efforts illustrated how computation could surpass human limitations, laying the foundation for a new era of machine-assisted problem solving.

2.3.2 Post-War Expansion: From Military to Commercial Computing

In the aftermath of the war, computing technologies rapidly extended into academic research, industrial automation, and eventually the consumer market. The invention of the **transistor** in 1947, followed by the development of **integrated circuits**, enabled a dramatic reduction in size, cost, and power consumption of computing devices.

By the 1970s and 1980s, companies such as **IBM**, **Apple**, and **Microsoft** were instrumental in the democratization of computing. Their contributions led to the widespread adoption of **personal computers**, transforming what was once an industrial tool into an everyday instrument for work, education, and entertainment.

2.4 The Birth of the Internet: From Military Networks to Global Infrastructure

Despite the growing capabilities of standalone computers, a true revolution emerged with the advent of **networked computing**. Initiated in the late 1960s, the **ARPANET** project—funded by the U.S. Department of Defense—introduced the concepts of **decentralized communication** and **packet switching**, allowing data to be transmitted in more reliable and efficient ways.

The subsequent standardization of **TCP/IP protocols** in the early 1980s enabled the seamless interconnection of disparate networks, laying the groundwork for the modern **Internet**. The introduction of the **Domain Name System (DNS)** in 1984 further simplified access to networked resources by replacing numeric IP addresses with human-readable names.

A transformative milestone occurred in 1991, when **Tim Berners-Lee** developed the **World Wide Web**, a hyperlinked system that provided a user-friendly interface for navigating online information. This innovation catalyzed a massive expansion in digital communication, turning the internet into a **global infrastructure** by the mid-1990s—enabling not only communication and commerce, but also the emergence of **competitive digital ecosystems** where individuals and organizations could engage, create, and compete on an unprecedented scale.

2.5 The Digital Arena: From Computing to Competition

As computational capabilities expanded and network infrastructures matured, new forms of competition began to emerge within digital spaces. Informal gaming gatherings—held in LAN parties and cybercafés—provided the cultural groundwork for organized digital play. These early communities, driven by curiosity and technical experimentation, laid the foundation for a broader transformation in how games were perceived and played.

The rise of online gaming redefined digital entertainment as a structured and competitive experience. The introduction of **matchmaking systems**, **ranking ladders**, and **persistent progression mechanics** transformed casual play into continuous skill-based challenges. Pioneering titles such as **Quake**, **Counter-Strike**, and **StarCraft** set the standard for modern competitive games, combining reflex-based gameplay with strategic depth.

These systems offered players clear feedback loops—performance metrics, leaderboards, win/loss ratios—that fueled their desire to improve and dominate. As a result, gaming began to evolve from a leisure activity into a compelling arena of personal achievement and social recognition.



Figure 2.1: LAN party in the late 1990s—a grassroots precursor to modern esports.

2.6 From Networks to Virtual Battlefields

The transformation of video games into **virtual battlefields** was fueled by technological advancements, global connectivity, and the rise of online communities. With real-time multiplayer becoming the norm, players worldwide could compete instantly, honing tactics and strategies to achieve recognition.

Video games offered an ideal competitive environment: **clear rules**, **quantifiable performance**, and **equal access**. As participation expanded, so did the supporting infrastructure—grassroots tournaments evolved into professional leagues, and casual players into sponsored competitors.

This convergence of entertainment, technology, and sport gave rise to **esports**, a global industry with professional teams, media coverage, and multimillion-dollar prize pools. What began as leisure became a legitimate career path and a powerful cultural force.

The same instinct that once ensured survival now drives mastery in digital arenas, where **technical skill**, **cognitive agility**, and **strategic thinking** define success.

The next chapters examine how this competitive landscape has also fostered new forms of rule-breaking. We will explore the evolution of **cheating** as both a technical challenge and a cultural phenomenon, revealing how players manipulate systems designed to ensure fair play.

3. THE EVOLUTION OF CHEATING: FROM DEBUGGING TOOLS TO AN ORGANIZED INDUSTRY

The history of cheating in video games is deeply intertwined with the evolution of the industry itself. What began as debugging tools for developers gradually transformed into a clandestine culture of modifications, and with the advent of online gaming, an organized and profitable industry with ties to cybercrime [4]. Simultaneously, the fight against cheating led to the development of **anti-cheat systems**, which evolved from simple integrity checks to sophisticated solutions incorporating artificial intelligence and deep system access [5].

1 The Early Days: Debugging Tools and Experimental Modifications

Before video games became a mainstream form of entertainment, early cheating emerged as a necessity within software development. In the 1970s and 1980s, developers embedded debugging tools within their games to facilitate testing. These systems allowed programmers to modify internal values such as lives, ammunition, and speed, enabling them to test different scenarios without playing through the entire game [6].

Some of these tools remained accessible in commercial releases, either unintentionally or as a deliberate choice by developers. This led to the creation of the first **cheat codes**, which were key combinations or commands that altered gameplay. One of the most iconic examples is the **Konami Code** (↑ ↑ ↓ ↓ ← → ← → B A), introduced in *Gradius* (1986) and later popularized in *Contra* (1988) [7]. While these codes were not designed to affect competition, they set a precedent for the intentional modification of gameplay.

2 The Rise of Cheating in Consoles and PC

As the video game industry grew throughout the 1980s and 1990s, cheating expanded beyond debugging tools and built-in cheat codes. With the advent of home consoles, external devices specifically designed to modify gameplay began to appear.

2.1 Cheat Devices: Game Genie, GameShark, and Pro Action Replay

The first dedicated cheat devices were **modification cartridges**, such as the **Game Genie** (1990) and the **GameShark** (1995). These hardware add-ons allowed players to inject custom modifications into a game's memory, altering key parameters such as health, ammunition, and physics [8]. While initially marketed as harmless tools for enhancing single-player experiences, they introduced the concept of **real-time game manipulation**, which would later be adapted for online environments.

On the PC side, the introduction of **memory editors** provided users with a more flexible and powerful way to modify game values at runtime. Early programs like **Cheat Engine** (2000) enabled players to search for specific in-game variables and alter them directly, bypassing the limitations of built-in cheat codes [9]. This shift from **developer-sanctioned cheats** to **unauthorized external modifications** laid the foundation for more advanced game-hacking techniques.

As gaming technology evolved, so did the tools available for modifying game memory structures. More sophisticated programs such as **ReClass.NET** emerged, allowing users to analyze and modify game memory at a structural level. Unlike traditional memory scanners, ReClass.NET enables users to reverse-engineer game memory layouts by dynamically inspecting class structures and data offsets in real-time [10]. This advancement significantly improved the ability of cheat developers to create complex modifications, from automatic aimbots to more intricate exploits that manipulate how game logic processes data.



Figure 3.1: Game Genie

These tools, originally designed for educational and debugging purposes, soon became widely used in cheat development, particularly in online gaming environments. Their increasing complexity meant that anti-cheat systems had to adapt rapidly, leading to an ongoing technological arms race between cheat developers and security teams.

3 The Transition to Online Gaming: Cheating Becomes a Competitive Threat

The rise of online multiplayer gaming in the late 1990s and early 2000s fundamentally changed the impact of cheating. Unlike single-player cheats, which only affected the individual experience, **online cheating directly disrupted fair competition** [5]. As games like **Quake** (1996), **Counter-Strike** (1999), and **Diablo II** (2000) introduced match-making and ranking systems, the incentive to cheat increased dramatically [11].

The growing importance of competitive integrity led developers to take more aggressive measures against cheating, resulting in the creation of **server-side and client-side anti-cheat mechanisms**. However, as anti-cheat solutions became more sophisticated, so too did cheating techniques, leading to an ongoing technological arms race between game security teams and cheat developers.

This shift led to the development of the first organized cheat communities, where players and programmers shared exploits, scripts, and modified game clients. It also marked the beginning of an **arms race** between cheat developers and game security teams, as new forms of cheating emerged alongside increasingly complex anti-cheat measures.

4 The Birth and Evolution of Anti-Cheat Systems

As online gaming became increasingly competitive, the integrity of fair play was challenged by the widespread use of cheats. Unlike single-player modifications, which had no broader consequences, cheating in multiplayer environments compromised **match-making, rankings, and esports competitions**, creating frustration among players and damaging the credibility of competitive gaming.

Early attempts to mitigate cheating relied on **server-side validation**, where game servers monitored player actions for anomalies that suggested external manipulation. These checks aimed to detect unnatural behavior, such as **impossible movement speeds, improbable accuracy, and modifications to game assets** that altered visibility constraints. However, this approach had significant limitations, as many cheats operated entirely on the **client side**, modifying game data before it was transmitted to the server. As a result, more sophisticated **client-side anti-cheat solutions** were developed to monitor and detect unauthorized modifications directly within the player's system.

One of the first dedicated anti-cheat programs was **PunkBuster**, developed by Even Balance, Inc. and introduced in **2000**. Initially implemented in *Return to Castle Wolfenstein* and later in *Battlefield 1942*, PunkBuster marked a shift from basic server-side detection to **active client-side monitoring**. It worked by scanning a player's system for **known cheat signatures**, verifying **game file integrity**, and capturing periodic **in-game screenshots** to identify visual modifications such as **wallhacks**. Additionally, PunkBuster maintained a **global ban list**, preventing known cheaters from accessing protected servers. Despite these advancements, **cheat developers quickly adapted** by employing techniques such as **code obfuscation**, allowing their software to remain undetected. PunkBuster also faced criticism for its **aggressive scanning approach**, which occasionally resulted in **false positives**, banning legitimate players due to conflicts with background applications.

In **2002**, Valve introduced **Valve Anti-Cheat (VAC)** alongside *Counter-Strike 1.6*, implementing a more **automated and data-driven approach** to cheat detection. Unlike PunkBuster, which **immediately expelled** suspected cheaters, VAC introduced a **delayed banning system**. This strategy allowed suspected cheaters to continue playing while logging their infractions, making it harder for cheat developers to **identify detection triggers** and modify their cheats accordingly. In addition to **signature-based detection**, VAC utilized **heuristic analysis** to recognize **abnormal behavioral patterns**, refining its detection algorithms over time. By integrating **server-side validation** with **machine learning techniques**, VAC continuously improved its ability to detect and counteract evolving cheats. However, since it **operated at the user level** rather than the **kernel level**, advanced cheats that manipulated system memory with higher privileges could still **bypass its protections**.

As **cheat developers refined their evasion techniques**, more **aggressive anti-cheat solutions** emerged. One of the most notable advancements was **BattleEye**, introduced in **2004** and later adopted in games like *Arma 2*, *Rainbow Six Siege*, and *PUBG*. Unlike VAC, which prioritized **passive detection**, BattleEye enforced **stricter security measures** by running with **elevated system privileges**. By operating at the **kernel level**, it could **detect and block cheat software** that manipulated **game memory at a deeper level**, making it significantly harder for external programs to **alter in-game data undetected**. This approach allowed BattleEye to **intercept and prevent unauthorized modifications** before they could affect gameplay, offering a **more proactive defense** against sophisticated cheats.

As **competitive gaming and esports** continued to grow, **Riot Games** introduced a new anti-cheat solution, **Vanguard**, designed specifically for *Valorant*. Released in **2020**, Vanguard took **anti-cheat enforcement to an unprecedented level** by implementing **persistent kernel-level monitoring**, meaning it **actively ran as a background process from the moment the operating system started**. This deep system integration enabled it to **detect and prevent unauthorized modifications** at the **lowest levels of system memory**, making it one of the **most invasive yet effective anti-cheat**

solutions. However, its **intrusive nature sparked controversy** among players concerned about **privacy**, as the software had **continuous access to system processes even when the game was not running.**

The **evolution of anti-cheat systems** reflects an **ongoing technological arms race** between **cheat developers and game security teams.** While early efforts relied on **rudimentary server-side validation**, modern solutions integrate **behavioral analysis, machine learning, and kernel-level access** to provide more robust protection. Despite these advancements, the constant adaptation of cheat developers ensures that no anti-cheat system remains **impenetrable for long**, perpetuating a **continuous cycle of exploitation and countermeasures** in the digital battlefield of competitive gaming.

This historical evolution has given rise to a wide variety of anti-cheat implementations, each shaped by the needs and challenges of its era.

The continuous development of anti-cheat solutions over the past two decades can be summarized through a timeline of key technologies. Figure 3.2 provides a chronological overview of the most influential anti-cheat systems, from early client-side detectors to modern kernel-level and behavior-based solutions.

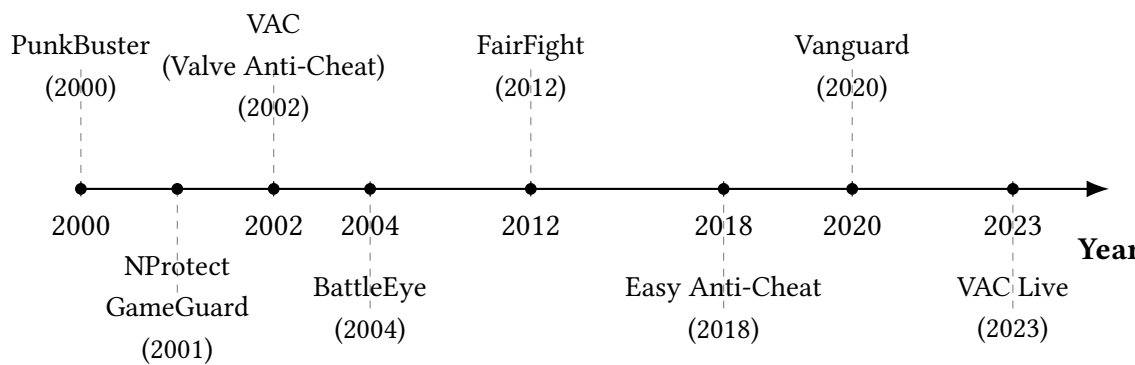


Figure 3.2: Chronological overview of key anti-cheat systems introduced between 2000 and 2023.

Among the systems depicted in the timeline, several stand out for their unique approach and impact.

nProtect GameGuard, launched in 2001, was one of the first anti-cheats to adopt rootkit-like behavior. Widely used in Asian MMORPGs, it embedded itself deeply in the operating system to prevent cheats from interacting with game processes, although this also raised concerns about privacy and system stability.

FairFight, introduced in 2012, departed from traditional detection mechanisms by using server-side statistical analysis to flag abnormal player behavior. Rather than inspecting memory or processes, it relied on gameplay metrics such as accuracy, reaction time, and movement patterns—making it less intrusive, but also vulnerable to both false positives and high-skill players.

Easy Anti-Cheat (EAC), which gained traction in 2018, became a popular choice for fast-paced competitive games. It combines kernel-level access with performance-friendly design and supports real-time detection of unauthorized drivers, injected DLLs, and tampering attempts—balancing robustness and user transparency.

Finally, **VAC Live**, unveiled in 2023, represents Valve’s shift toward real-time, data-driven cheat prevention. Unlike the original VAC system, which relied on delayed bans, VAC Live aims to detect and react to cheats during gameplay, making it a critical component in safeguarding the integrity of high-stakes esports matches.

5 The Commercialization of Cheating

The rapid commercialization of cheating has transformed it from a niche activity into a highly organized industry, offering both software-based and hardware-level exploits. Specialized forums such as **UnKnoWnCheaTs**, **MPGH**, and **ElitePVPers** have become central hubs for distributing cheats, providing resources for reverse engineering, memory manipulation, and anti-cheat evasion. These platforms, along with encrypted messaging services like **Discord** and **Telegram**, have facilitated the sale and distribution of sophisticated cheats through subscription models and private communities.

Table 3.1 summarizes some of the most prominent cheat providers currently active in the CS2 ecosystem.

Provider	Released	Subscription Price	Access Model
Neverlose [12]	2020	€17.10/month, €44.10/3 months, €134.10/year	Invite-only
Memesense [13]	2023	\$4/14 days, \$8/month, \$12/3 months	Public access
Fatality.win [14]	2022	Premium pricing (variable)	Invite-only
Project Infinity [15]	2017	€14.99/month, €29.99/3 months, €99.99 lifetime	Public access
Midnight [16]	2019	£5.42/month	Public access
Onetap [17]	2024	Pricing not publicly disclosed	Public access
Skeet (Game-sense) [18]	2016	\$16/month (via tokens, fluctuating)	Invite-only + HWID locked

Table 3.1: Overview of major CS2 cheat providers, including release year, pricing models, and access restrictions.

As cheat developers adopt increasingly advanced techniques, such as kernel-level drivers and DMA-based exploits, anti-cheat systems are forced into a continuous state of adaptation. The emergence of software-based cheats, particularly those using DLL injection, code hooking, and API interception, has proven to be one of the most adaptable and challenging threats. These methods allow cheat developers to modify game functions dynamically, bypassing traditional detection systems.

From History to Implementation: A Technical Transition

Having traced the historical and sociotechnical evolution of cheating—from its roots in debugging tools to its current status as a commercialized, clandestine industry—we now shift focus toward the technical foundations that sustain it today.

The next chapters will abandon the literary and historical perspective in favor of a rigorous, low-level examination of how modern software-based cheats are constructed and deployed. We will explore the inner mechanics of code injection, memory manipulation, and anti-cheat evasion techniques, providing a detailed view of the technological arms race from a reverse engineering and systems programming standpoint.

With the historical context established, it is time to dissect the machinery.

4. THE PREVALENCE AND IMPACT OF CHEATING IN ONLINE MULTIPLAYER GAMES

Cheating in online multiplayer games constitutes an escalating systemic threat that undermines fairness, erodes player trust, and destabilizes gaming ecosystems. This chapter examines the multifaceted consequences of cheating, with particular emphasis on the evolving landscape surrounding *Counter-Strike 2* (CS2), and argues that addressing this issue demands an integrated response encompassing technical, legal, and social counter-measures.

As competitive online titles continue to thrive—especially those with ranked ladders, esports ecosystems, and in-game economies—the incentives to obtain unfair advantages intensify. This dynamic perpetuates a relentless arms race between cheat developers and game studios, requiring not only robust technological defenses but also a broader understanding of the socio-economic impacts of cheating.

1 Cheating as a Critical Threat to Player Retention

Cheating is far from a minor inconvenience; it is a leading driver of player attrition and dissatisfaction. A global survey by Irdeto found that 60% of players experienced a negative impact on their enjoyment due to cheating, and 77% indicated they would abandon a game if cheating became pervasive 19.

Table 4.1: Survey Results on Cheating in Online Games (Irdeto, 2018)

Survey Statement	Percentage of Respondents
Experienced negative impact due to cheating	60%
Would stop playing a game because of cheaters	77%
Support permanent bans for cheaters	69%
Believe developers are not doing enough	55%

These findings underscore that cheating not only generates immediate frustration but also carries long-term economic implications. Since engagement drives monetization through battle passes, cosmetic sales, and advertising, a decline in player retention fueled by cheating poses a direct threat to publishers’ revenue streams.

Ultimately, when players perceive ranked systems, matchmaking, or competition outcomes as compromised, they are likely to migrate toward alternative titles—thus undermining the sustainability of the gaming community.

2 Case Study: Cheating in *Counter-Strike 2*

Counter-Strike 2 (CS2), as one of the most prominent competitive FPS titles, provides an instructive case study of modern cheating dynamics. Its large player base and vibrant esports scene render it both a lucrative target for cheat developers and a crucial testing ground for anti-cheat innovations.

To contextualize the scale of CS2’s ecosystem, Figure 4.1 presents the evolution of its player base based on SteamCharts data, covering the period from January 2024 to April 2025:

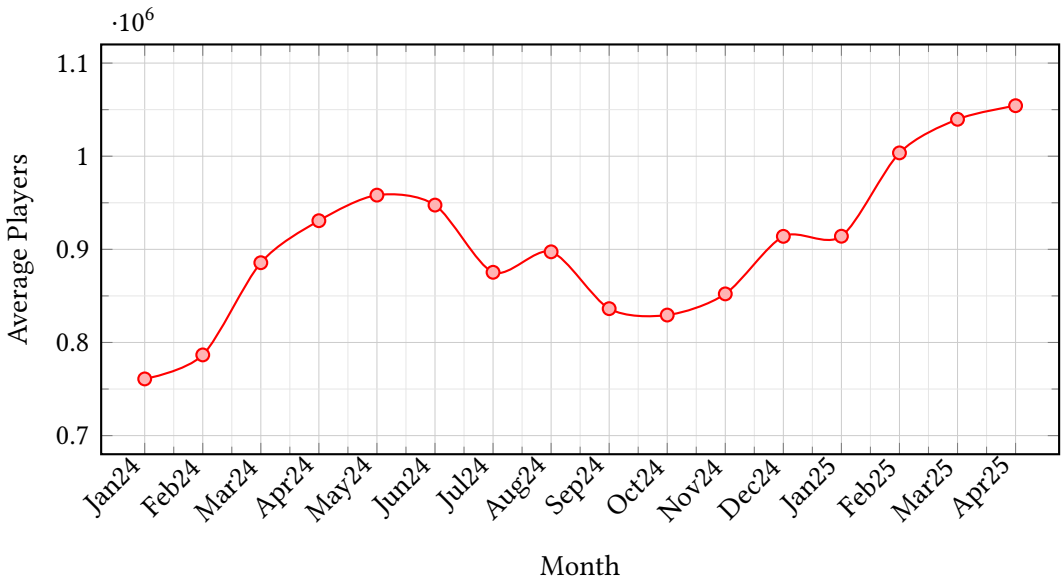


Figure 4.1: Evolution of Average CS2 Player Counts (Jan 2024 – Apr 2025). Source: SteamCharts.

The steady growth in CS2’s player base highlights the game’s enduring popularity and its pivotal role in the competitive FPS scene. Yet, with expansion comes increased complexity: a larger and more heterogeneous player community makes preserving **competitive integrity** a significantly greater challenge.

Despite notable improvements to the Valve Anti-Cheat (VAC) system and the *Overwatch* platform, modern cheats—employing *polymorphic code*, *manual injection*, and *kernel-level techniques*—enable many offenders to operate undetected for prolonged periods. The result is an ongoing **arms race** between cheat developers and anti-cheat engineers, requiring constant innovation to safeguard fair play.

Data from recent enforcement activity further underscores the scale and persistence of the issue:

Table 4.2: Activity of Banned Accounts Prior to Enforcement (Jan–Apr 2024)

Metric	Estimated Value
Average delay between last match and VAC ban	20 days
Percentage of bans issued within 24 hours	28%
Total matches played by banned accounts	32,000+
Percentage of matches with only one banned player	97%
Evidence of case farming behavior	Not significant

These observations reveal that cheaters are not isolated anomalies; rather, they are embedded within regular matchmaking sessions, further complicating the task of detection and enforcement.

2.1 Trust Factor and Reporting System Limitations

To mitigate cheating and improve matchmaking, Valve introduced the *Trust Factor* system—an opaque internal score assessing player reliability.

Trust Factor Explained

The Trust Factor metric aggregates multiple indicators—including VAC bans, player reports, community standing, and system configurations—to invisibly adjust matchmaking quality. Its secrecy aims to prevent gaming the system but inadvertently introduces confusion and exploitation.

While the Trust Factor system offers a promising foundation, its lack of transparency exposes two critical vulnerabilities: (1) Players have limited avenues for understanding or improving their score; (2) Third-party services offering *report bots* and *commend bots* manipulate trust metrics for profit.

2.2 Economic and Psychological Impact

The consequences of cheating extend beyond gameplay to inflict significant economic and psychological harm.

In early 2025, a major VAC enforcement wave permanently banned accounts with an estimated combined inventory value exceeding \$2 million **20**. Although hailed as a success against cheaters, it also fueled concerns regarding transparency and false positives.

Definition: VAC Ban

A Valve Anti-Cheat (VAC) ban is a permanent restriction applied when cheating activity is detected. It disables multiplayer functionality and trading privileges associated with the affected title.

For players who have invested considerable time, resources, and personal attachment into their accounts, the psychological toll of either encountering cheaters or being wrongfully penalized can be profound—leading to disengagement and a decline in community loyalty.

3 Community Perception and the Culture of Cynicism

Cheating incidents are highly visible across social platforms such as Reddit, YouTube, and Twitch. High-profile streamers encountering cheaters during broadcasts exacerbate perceptions of widespread dishonesty, even when empirical cheating rates remain relatively low.

This feedback loop fosters a culture of cynicism, where player trust in matchmaking systems and anti-cheat technologies diminishes—ultimately endangering long-term community stability.

4 Legal and Ethical Responses to Cheat Development

Recognizing the limitations of technical interventions, developers increasingly pursue legal remedies against cheat creators.

Notably, Bungie’s \$12 million victory against AimJunkies in 2023 established a strong legal precedent. Subsequently, companies such as Riot, Ubisoft, and Activision have adopted similar strategies.

Beyond legality, cheat distribution undermines the ethical foundations of competitive gaming, corrupting merit-based progression and threatening the credibility of esports ecosystems.

5 Anti-Cheat Technologies: Current Trends and Limitations

Despite considerable innovation, anti-cheat technologies remain primarily reactive.

Valve’s VAC system relies heavily on signature detection, rendering it vulnerable to obfuscation and manual mapping techniques. In contrast, Riot’s Vanguard leverages kernel-level access but provokes privacy concerns among users.

Emerging approaches—including AI-driven behavioral models and secure enclave technology—offer promising avenues but introduce new challenges, such as avoiding false positives and preserving user privacy.

Thus, the technological arms race between cheat developers and anti-cheat engineers remains a dynamic and relentless struggle.

6 Toward a More Resilient Competitive Ecosystem

Addressing the systemic threat of cheating requires a comprehensive, multi-layered strategy encompassing technical, legal, and social dimensions.

Key initiatives include:

- Implementing real-time, behavior-driven detection systems.
- Publishing regular transparency reports on enforcement outcomes.
- Enhancing user feedback and appeal mechanisms for systems like Trust Factor.
- Educating players on fair play and its long-term benefits.
- Pursuing aggressive legal action against cheat developers and distributors.

Without sustained reform and vigilance, even the most iconic multiplayer ecosystems risk erosion of integrity and the consequent decline of player engagement.

Key Takeaways:

- Cheating directly threatens player retention, game reputation, and revenue.
- Systems like Trust Factor must evolve with greater transparency.
- Legal action constitutes an essential front against organized cheat development.
- The fight against cheating is an ongoing, adaptive arms race requiring continual innovation.

5. INJECTION AND DETECTION TECHNIQUES

1 Why Injection Matters in Cheat Development

In professional game cheating, especially in competitive games like *Counter-Strike 2*, most serious cheats are built as **DLLs** that are injected into the game's running process.

What is a DLL?

A DLL (Dynamic Link Library) is like a small add-on that can be loaded into a program to extend what it can do. In the case of cheats, a DLL gives direct access to the game's memory and functions.

Injecting a DLL allows the cheat to run **at the same time** as the game itself. This enables cheats to:

- **Hook game functions** to change behavior (e.g., aiming or vision systems).
- **Modify memory** to adjust gameplay (e.g., speed, ammo, recoil).
- **Read internal data** like player positions or weapon stats.
- **Draw overlays** directly on the screen (e.g., radar, ESP).
- **Automate actions** to perform tasks faster or more accurately.

Because of this deep integration, cheat injection is the main focus of anti-cheat systems like **Valve Anti-Cheat (VAC)**.

2 Overview of Code Injection Techniques

Code injection means inserting new code into a running program. In cheat development, this usually means adding a DLL or custom code into the game's memory.

Traditional Methods

Earlier cheats used simple Windows functions like:

- `LoadLibrary`: Loads a DLL into another process.
- `CreateRemoteThread`: Starts a new thread inside another program.

These methods are now easy for anti-cheats to detect because they leave obvious traces.

Manual Mapping

Today, cheats mostly use a method called **Manual Mapping**.

What is Manual Mapping?

Manual Mapping is a way of loading a DLL manually into a program's memory without using Windows' normal loading functions. This makes it much harder for anti-cheat systems to notice the injection.

Manual Mapping avoids showing up in the list of loaded modules and can start execution in stealthy ways, such as by hijacking existing game threads.

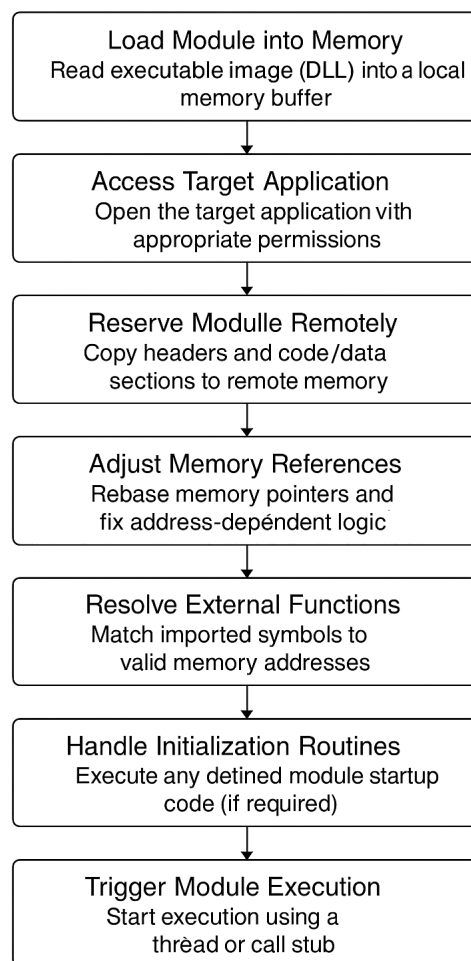


Figure 5.1: Simplified flowchart of dynamic module injection using Manual Mapping.

We have developed our own injector that follows the structure outlined in Figure 5.1. The source code for this injector can be found in the `injector` branch of the project repository.

3 Detection Techniques in Anti-Cheat Systems

The detection techniques described in this section are **not merely theoretical**; they are the result of **practical reverse engineering** carried out on modules *extracted (dumped)* directly from the game's memory during runtime. Through detailed analysis of the internal workings of **Valve Anti-Cheat (VAC)**, we have gathered **concrete evidence** supporting the mechanisms and strategies presented below.

Modern anti-cheat systems, such as VAC, deploy a wide range of methods to detect the presence of unauthorized code within game processes. These methods are specifically designed to identify *abnormal memory modifications*, *injected modules*, or *manipulation of trusted system structures*.

In the following sections, we outline and explain the principal detection techniques observed during our research.

Thread Local Storage (TLS) Callbacks

One early opportunity for anti-cheats to detect injected code occurs during program startup. When a program or DLL starts running, Windows can automatically execute special functions known as **TLS callbacks**.

What is a TLS Callback?

A TLS (Thread Local Storage) callback is a special function that Windows automatically calls when a thread starts, before the main application code runs. Developers can use it for initialization tasks — but it can also be leveraged by anti-cheats for early memory inspections.

Anti-cheats leverage this early execution stage to scan memory and inspect the runtime environment, often before the main game code even begins.

If an injected DLL includes TLS callbacks and they are not properly removed or disabled, the cheat risks being detected immediately during process initialization.

Start Address Integrity Checks

Beyond startup checks, anti-cheats like VAC continuously monitor thread creation during gameplay. A common detection method involves checking the **start address** of each thread.

What is a Start Address?

The start address is the location in memory where a thread begins its execution. Legitimate threads typically start inside official game modules, such as `client.dll` or other trusted libraries.

If a thread is found starting at an address like `0x5F3A0000`, which falls outside any known game module (e.g., not within `client.dll`), it is highly suspicious and often indicates injected code or shellcode execution.

This method is particularly effective at catching poorly concealed code injections or shellcode execution.

VMT Hook Detection

Another detection strategy focuses on the integrity of in-game objects.

In object-oriented engines, game entities typically use **Virtual Method Tables (VMTs)** to organize their functions.

What is a Virtual Method Table (VMT)?

A VMT is a table used in object-oriented programming to map object methods to their actual function addresses. By modifying a VMT, an attacker can hijack how a game object behaves.

Cheats often modify these tables to hijack behaviors, such as altering rendering or player input.

Anti-cheats like VAC can identify tampered VMTs by verifying:

- Whether the VMT points to unexpected memory regions.
- Whether the methods referenced fall outside legitimate game or system modules.

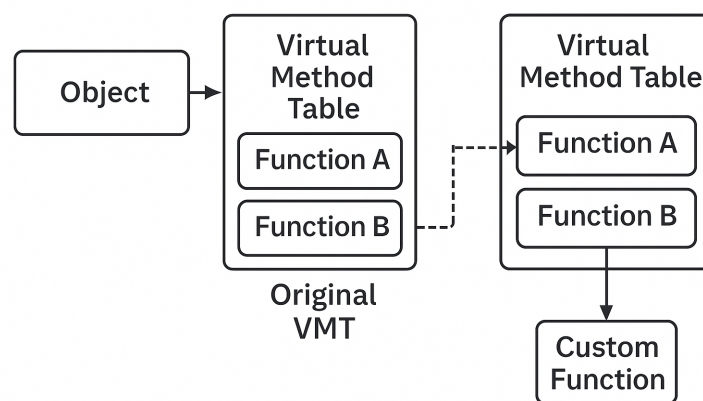


Figure 5.2: Example of VMT Hooking.

Due to their clear memory footprint, traditional VMT hooks are now considered relatively high-risk in modern cheat development.

4 Evasion Techniques Used by Cheats

As anti-cheat systems like VAC become more sophisticated in their detection capabilities, cheat developers continuously adapt by designing increasingly advanced evasion techniques. These strategies aim to minimize the attack surface, avoid triggering heuristic scans, and maintain stealth even under active monitoring.

In this section, we present and explain some of the most common methods used to evade detection.

Suppressing TLS Callbacks

To avoid early-stage detection, cheats often remove or disable the TLS callback directory from their injected DLLs. By doing so, they prevent anti-cheats from executing inspection routines before the cheat is fully concealed.

Thread Hijacking and Return Address Spoofing

Instead of creating new threads—an action easily spotted—some cheats **hijack existing game threads**.

What is Thread Hijacking?

Thread hijacking involves pausing an existing, legitimate thread, changing its instruction pointer to execute custom code, and then resuming it—making it harder for anti-cheats to notice any new activity.

To further hide execution, cheats use **return address spoofing**—manipulating the call stack to make it appear that their code was naturally invoked from within the game's legitimate modules.

Start Address Redirection via ROP Gadgets

A more advanced and stealthy method used by cheats involves **Return-Oriented Programming (ROP)**.

What is a ROP Gadget?

A ROP gadget is a small snippet of code already present in trusted modules, such as the game executable or Windows libraries. These snippets typically end with control transfer instructions like `ret` (return) or `jmp reg`.

Instead of directly calling malicious code (which could be easily detected), cheats use ROP gadgets to build fake call chains. This way, their execution path appears legitimate, making detection much harder.

Stack Alignment and Shadow Space. In 64-bit Windows environments, the calling convention (Microsoft x64) imposes strict rules on how functions must be called, particularly regarding the stack layout:

- **Stack Alignment:** Before calling any function, the stack pointer (rsp) must be 16-byte aligned. This ensures that SIMD instructions (like movaps) and internal ABI requirements function correctly.
- **Shadow Space:** Additionally, 32 bytes must be reserved on the stack for the called function to use freely. This space is used to store register parameters or local variables.

Failure to meet these requirements can cause functions to crash, corrupt data, or behave unpredictably. Moreover, violating calling conventions can raise suspicion in anti-cheat systems performing runtime integrity checks.

To correctly prepare the stack for a spoofed or redirected call during ROP, cheats must adjust the stack before jumping to the target function.

Here is an example of a minimal x64 call stub that correctly aligns the stack and reserves the required shadow space:

```
1  sub    rsp, 0x28                ; reserve 32 bytes shadow space + align
   ; stack (16-byte alignment)
2  mov    rax, 0xFFAAFFAAFFAAFFAA ; address of the ROP gadget to act as a fake
   ; return address
3  push   rax                     ; push the fake return address onto the
   ; stack
4  mov    rax, 0xFFAAFFAAFFAAFFAA ; address of the actual target function to
   ; execute
5  jmp    rax                     ; jump to the real function
```

This setup ensures that:

- The stack is properly aligned (after `sub rsp, 0x28`).
- A valid-looking return address (pointing to a trusted ROP gadget) is pushed.
- The function call structure looks legitimate, reducing the risk of detection.

Thus, when the called function finishes and executes a `ret` instruction, it returns to a safe, believable address instead of crashing or exposing the cheat.

Detection Vectors. While ROP techniques are highly stealthy, anti-cheats can still perform **stack walking**—analyzing the call stack at runtime to detect inconsistencies, fake return addresses, or suspicious memory regions.

Practical Implementations. Open-source projects like x86RetSpoof by danielkrupinski offer practical examples of return address spoofing. However, professional cheats often use customized ROP implementations tailored to specific games like CS2 for maximum stealth.

5 Extended VAC Capabilities and Developer Countermeasures

Beyond basic memory and thread inspections, **Valve Anti-Cheat (VAC)** extends its detection efforts through broader system monitoring. It actively scans the file system looking for known cheat artifacts, inspects running processes for suspicious memory regions or injected modules, monitors the Windows registry for abnormal entries, and tracks inter-process activity to detect unauthorized access to the game.

This expanded approach allows VAC to move beyond simple memory analysis, building a more complete picture of system behavior and making traditional static cheats easier to detect.

In response, cheat developers have introduced a range of countermeasures aimed at staying hidden. String encryption is commonly used to obfuscate identifiable patterns in memory. Name randomization changes module and window identifiers dynamically to avoid blacklists. More advanced techniques include polymorphic code, which mutates the cheat at runtime to evade pattern detection, and fileless execution, where the cheat operates entirely from memory without touching the disk. Memory cleansing after injection further minimizes visible traces that could expose the cheat during active scanning.

Ultimately, the ongoing battle between anti-cheat technologies and cheat developers drives constant innovation on both sides. As VAC strengthens its methods to detect unauthorized activity, cheats evolve to become more stealthy, adaptive, and resilient — ensuring that the arms race in competitive gaming security remains in constant motion.

6. FUNCTION HOOKING AND TRAMPOLINE-BASED INTERCEPTION

Overview

At the heart of our cheat framework lies a fundamental and widely-used technique: **function hooking**. This mechanism allows us to intercept and manipulate internal function calls within the game, modify their behavior, and seamlessly resume execution without disrupting the application's control flow.

What is Function Hooking?

Function hooking is the practice of redirecting a function's execution flow to custom code. It enables the interception, modification, or extension of a program's behavior without access to its source code.

Among the various hooking strategies available, the most stable and stealthy is the **trampoline hook**. Trampoline hooking offers a reliable and elegant solution for redirecting code execution to custom payloads while preserving the original function's logic.

In this chapter, we present an in-depth explanation of trampoline hooks—how they work, why they are effective, and how they can be implemented safely using the MinHook library.

1 What is a Trampoline Hook?

A trampoline hook works by overwriting the beginning of a target function with a jump (typically a near relative `JMP` or an absolute indirect jump) to a custom handler — our hook payload. Since this patch modifies the first few bytes of the original function, those instructions must first be preserved to avoid corrupting the original behavior. We do this by copying them into a new, executable memory region known as the *trampoline*.

What is a Trampoline?

A trampoline is a small section of executable memory that stores the original stolen instructions from a hooked function, allowing seamless continuation after custom code executes.

Once our custom payload finishes, control passes to the trampoline, which executes the preserved instructions and then jumps back to the original function — resuming normal execution as if nothing had changed.

This redirection process involves four main components:

- **Original (Hooked) Function:** The function entry point is patched to redirect execution.
- **Relay Function:** A lightweight stub that jumps to the hook payload.
- **Hook Payload:** Executes the desired cheat logic before invoking the trampoline.
- **Trampoline:** Holds the stolen instructions and redirects execution back into the original code.

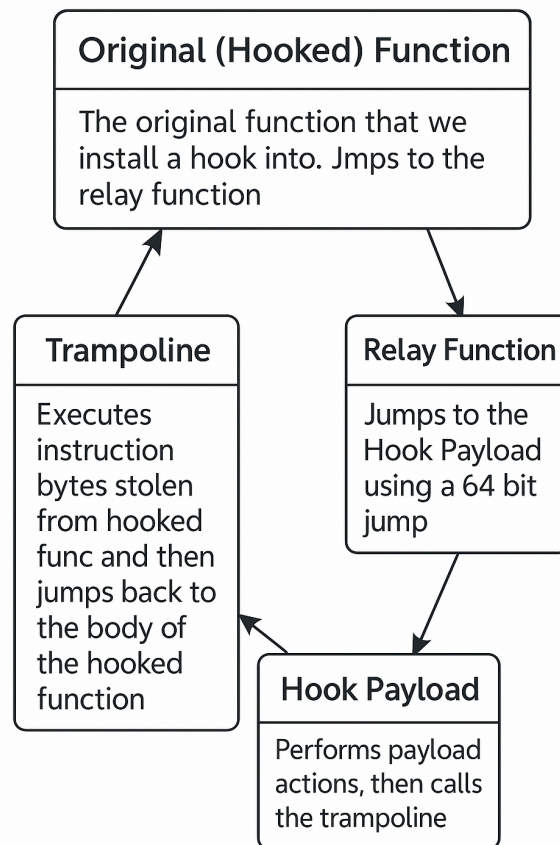


Figure 6.1: Control flow in a trampoline hook setup.

2 Reverse Engineering and Function Discovery

Before hooking any function, it is essential to identify which ones control the behaviors we want to alter. This requires reverse engineering the game’s dynamic libraries—most notably `client.dll`, which contains the majority of client-side logic.

Using static analysis tools such as *IDA Pro* and *Ghidra*, along with dynamic debugging tools like *x64dbg*, we analyzed exported symbols, disassembled machine code, and traced runtime behavior to locate key functions.

To overcome ASLR, we extracted **signatures** — short byte sequences unique to each function. Our injected DLL scans memory at runtime to locate these signatures and dynamically apply the hooks, ensuring compatibility across different sessions and system restarts. The implementation of the pattern scanning logic used for this purpose can be found in our codebase, specifically within the `helper.cpp` class.

Rather than scanning for static memory addresses—which are often relocated due to ASLR—we focus on scanning for **instruction bytes**. This approach ensures that our patterns remain valid across game updates and memory randomization. Typically, patterns include some wildcards (??) to account for dynamic elements such as relative offsets or pointers, which are not fixed between runs.

For instance, one of the patterns we use to locate the view matrix within `client.dll` is:

```
1 "48 8D ?? ?? ?? ?? ?? 48 C1 E0 06 48 03 C1 C3 CC CC"
```

Here, the ?? symbols act as **wildcards**, allowing any byte to match at those positions. This flexibility ensures that even if address operands change, the overall pattern remains detectable based on the surrounding instruction sequence.

3 Why Trampoline Hooks Are Effective

Trampoline hooks offer several distinct advantages for real-time game manipulation and anti-cheat evasion:

- They preserve the game's original control flow, allowing normal execution to continue after the cheat logic runs.
- They maintain stack integrity and valid return addresses, helping to evade detection techniques such as return address validation (used by VAC).
- They avoid introducing foreign code into new memory regions, reducing the risk of detection by memory scanners.

In short, trampoline hooking provides a balance between flexibility, stability, and stealth — making it the preferred method in both legitimate instrumentation tools and game hacking frameworks.

4 Implementing Hooks with MinHook

To install hooks safely, efficiently, and with minimal risk of destabilizing the game, we rely on **MinHook**—a lightweight, open-source hooking library for x86 and x64 Windows systems.

What is MinHook?

MinHook is a minimalistic and efficient library designed to simplify the process of function hooking. It automatically handles instruction relocation, trampoline creation, and thread-safe patching, offering a robust and reliable solution for both x86 and x64 applications.

Our hooking setup follows a clear structure:

- A typedef defines the original function's signature.
- A static pointer stores the original (trampoline) function.
- A static method implements the custom logic to be executed.

First, we define the structure for the hook:

```
1 // Class responsible for world color modulation hook
2 class WorldModulation {
3 public:
4     // Typedef for the original function signature
5     typedef void* (__fastcall* ModulateWorldColorFn)(...);
6
7     // Pointer to store the original (trampoline) function
8     static ModulateWorldColorFn oModulateWorldColor;
9
10    // Our custom hook function
11    static void* __fastcall hModulateWorldColor(...);
12 };
13
14 // Instance of the class
15 WorldModulation m_WorldModulation;
```

Installing the hook with **MinHook** is straightforward:

```
1 // Function to initialize and install the hook
2 void InitHook() {
3     MH_Initialize(); // Initialize MinHook library
4
5     // Create a hook: redirect TargetAddress to our custom function
6     MH_CreateHook((LPVOID)TargetAddress, &WorldModulation::
7     hModulateWorldColor, (LPVOID*)&WorldModulation::oModulateWorldColor);
8
9     MH_EnableHook((LPVOID)TargetAddress); // Enable the installed hook
10 }
```

The custom hook function applies a global color modulation if necessary:

```
1 // Custom function that replaces the original behavior
2 void* __fastcall WorldModulation::hModulateWorldColor(...) {
3     // Check if world modulation is enabled
4     if (!Globals::worldModulation)
5         return oModulateWorldColor(...); // If not, just call the
        original function
6
7     // Apply the global color to all world objects
8     for (int i = 0; i < pWorld->count; i++) {
9         auto& obj = pWorld->array[i];
10        obj.SetColor(Globals::worldModulationColor); // Apply the new
        color
11    }
12
13    // After custom logic, continue execution normally
14    return oModulateWorldColor(...);
15 }
```

This structure ensures that our custom logic only executes when necessary, preserving the original behavior otherwise.

5 Final Thoughts on Trampoline Hooking

Trampoline hooking provides a *stable*, *stealthy*, and *flexible* method for manipulating in-game behavior while minimizing the risk of detection or instability.

By carefully preserving **return addresses**, maintaining **control flow integrity**, and avoiding the introduction of foreign memory regions, trampoline hooks form the **foundation** of an effective and resilient cheat architecture.

The typical workflow begins with **reverse engineering** target DLLs to identify relevant functions, **generating dynamic signatures** for reliable location at runtime, and **installing hooks** using safe, battle-tested libraries like **MinHook**. This method ensures that our modifications remain compatible even as the game's memory layout evolves between sessions or updates.

In the next chapter, we will explore **advanced techniques** for identifying and prioritizing the most impactful functions to hook, particularly those related to *rendering*, *game logic processing*, and *player state management*.

7. INTERNAL GAME STRUCTURE: LOGIC BEHIND COUNTER-STRIKE 2

1 Understanding Entity Lists in Video Games

In modern game engines—particularly those designed for real-time, multiplayer environments—an **entity list** is a fundamental data structure responsible for managing all interactive objects within the game world. These entities include players, non-player characters (NPCs), projectiles, grenades, environmental props, and more.

What is an Entity?

An *entity* is any dynamic object in the game world, such as a player, NPC, projectile, or interactive prop, managed and updated by the engine each frame.

At the core, each entity is an instance of a class containing essential data such as position, health, team ID, state flags, and animation or input components. The engine maintains and updates these entities each frame to simulate gameplay logic, render visuals, process collisions, and evaluate state-based behavior such as death, visibility, or interaction.

1.1 Common Structures for Entity Lists

Entity lists are implemented using various structures, depending on the performance needs and memory model of the engine:

- **Array of Entity Objects:**
A flat, contiguous layout offering fast access, but limited flexibility.
- **Array of Pointers:**
A static array referencing dynamically allocated entities. Frequently used in older engine architectures.
- **std::vector:**
A dynamic container ideal for runtime creation and destruction of entities.
- **Linked List:**
A flexible memory structure where each node points to the next entity, at the cost of linear traversal time.
- **HashMap / Tree:**
Enables fast lookup and categorization of entities based on IDs or class types.

What is the Paged Entity System?

A memory management technique where entities are grouped into fixed-size pages, improving scalability, memory safety, and preventing invalid accesses when entities are destroyed or recreated.

While earlier iterations of the Source engine (such as CS:GO) utilized a simple flat array of entity pointers for entity management, the modern Source 2 engine in Counter-Strike 2 introduces a **paged entity system**. This design organizes entities into memory pages and utilizes safe handle-based referencing, offering greater scalability, safety, and efficiency across dynamically allocated objects.

2 Entity Organization in Counter-Strike 2

In *Counter-Strike 2*, every interactive object—whether a player, weapon, projectile, or environmental element—is internally represented as a dynamic **entity**. These entities are managed using unique *handles*, lightweight references that combine an entity index and a generation counter to ensure safe access and lifecycle management.

What is a Handle?

A *handle* is a compact reference to an entity, typically composed of an index and a generation counter. It allows efficient and safe resolution of entities, preventing issues such as dangling pointers when entities are destroyed and recreated.

This handle-based system enables robust referencing, safeguarding against invalid memory access even in highly dynamic environments where entities frequently appear and disappear during gameplay.

To map a handle to an actual entity in memory, the Source 2 engine employs a **two-level paging system**—a mechanism conceptually similar to virtual memory in modern operating systems. This approach balances memory efficiency, scalability, and safety, particularly as entity counts grow in large matches or complex game scenarios.

When analyzing player-specific data structures, two core classes emerge as critical pillars:

- **CBasePlayerController** — Manages high-level metadata about a player, including their sanitized name, Steam ID, current input state, and a handle linking to their in-world representation (the pawn).
- **C_CSPlayerPawn** — Represents the player's physical presence within the game world, storing vital gameplay attributes such as position, health, team affiliation, view angles, and animation states.

Both player-related classes inherit from **C_BaseEntity**, the core base class that provides common properties, behaviors, and networking support for all entities in the engine.

The following are simplified and annotated representations of these structures:

1. Base Entity – C_BaseEntity

```
1  /*****
2   * C_BaseEntity:
3   * Base class for all entities in Source 2.
4   *****/
5  class C_BaseEntity {
6  public:
7      CHandle<C_BaseEntity>    m_hEffectEntity;    // Effect entity
8      CHandle<C_BaseEntity>    m_hOwnerEntity;    // Owner
9      CHandle<C_BaseEntity>    m_hGroundEntity;    // Entity standing on
10     CBodyComponent::Storage_t m_CBodyComponent;  // Physics
11     int32_t                   m_iHealth;          // Entity health
12     Vector3                    m_vPosition;       // World-space
13     position
14
15     // ... Additional fields (e.g., team, animation state)
16 };
```

Listing 7.1: C_BaseEntity – Core Class for All Entities

2. Player Controller – CBasePlayerController

```
1  /*****
2   * CBasePlayerController:
3   * Manages metadata, Steam ID, input, and pawn handle.
4   *****/
5  class CBasePlayerController : public C_BaseEntity {
6  public:
7      std::string               m_sSanitizedPlayerName; // Player name
8      uint64_t                  m_steamID;              // Steam Id
9      InputState_t              m_inputState;           // Current input
10     CHandle<C_CSPlayerPawn>    m_hPawn;               // Handle to pawn
11     uint32_t                   m_iTeamNum;             // Team ID
12     bool                       m_bIsConnected;         // Connection
13     status
14
15     // ... Other metadata (e.g., score, latency)
16 };
```

Listing 7.2: CBasePlayerController – Player Metadata and Input Handler

3. Player Pawn – C_CSPlayerPawn

```
1  /*****
2   * C_CSPlayerPawn:
3   * Represents the physical player in the world.
4   *****/
5  class C_CSPlayerPawn : public C_BaseEntity {
6  public:
7      Vector3          m_vPosition;      // World position
8      uint32_t         m_iHealth;        // Health
9      uint32_t         m_iTeamNum;       // Team ID
10     QAngle            m_angViewAngles;  // View angles
11     AnimationState_t  m_AnimState;      // Animation state
12
13     // ... Other attributes
14 };
```

Listing 7.3: C_CSPlayerPawn – In-World Representation of the Player

This class hierarchy enables a clear separation of concerns between the *logical control* of the player (controller) and their *physical simulation* within the game world (pawn). This design also accommodates edge cases such as spectator modes, bot logic, and replay systems, where a controller may exist without an active pawn.

Moreover, the architecture follows a variant of the **Entity-Component-System (ECS) paradigm**, promoting modularity and flexibility. This pattern is especially beneficial for external tools such as mods, analytics systems, and cheat frameworks.

3 Entity List Traversal

Having established the structural foundations of player entities, the next step involves accessing and enumerating these entities during runtime. This process—known as **entity list traversal**—is essential for real-time game modification, data extraction, and the development of external tools.

3.1 Locating and Identifying the Entity List in Memory

In the absence of **debugging symbols** or **SDK access**—a common scenario in protected or closed-source games—discovering the location and structure of the **entity list** becomes a crucial part of the reverse engineering process. Two primary approaches are typically employed: *dynamic memory inspection* and *static binary analysis*.

These complementary methods allow external programs to interact systematically with the game’s internal structures.

Dynamic Approach: Real-Time Memory Inspection

Dynamic memory inspection focuses on observing changes in the game's memory while it is running, providing a practical way to locate the entity list without dissecting the entire executable.

The discovery process generally follows these steps:

1. **Scan for the Local Player's Position:** Search for changing float values that represent the player's coordinates (X, Y, Z) during movement.
2. **Filter Out Non-Relevant Addresses:** Discard addresses that stop updating when the game is paused or unfocused, as they typically reflect cached or rendering-only data.
3. **Analyze Accessing Instructions:** Use memory inspection tools to detect which instructions interact with these addresses.
4. **Detect Multi-Entity Patterns:** Identify if the same instruction accesses positions of multiple players, suggesting iteration over an entity container.
5. **Recover Entity Base Pointers:** Subtract known field offsets (e.g., `m_vPosition`) from accessed addresses to obtain base pointers to entities.
6. **Locate the Entity List Structure:** Search for contiguous memory regions holding these base pointers, checking for uniform spacing, characteristic of structured storage (arrays or paged layouts).
7. **Confirm and Generalize:** Build a traversal loop using pointer arithmetic to iterate through the list and access relevant entity attributes such as health, team ID, and state flags.

Important: Not all structures resembling entity lists are gameplay-relevant. Careful validation is necessary to avoid mistaking render-only data for true gameplay entities.

Types of Entity Lists Encountered

During memory analysis, it is typical to encounter two distinct categories of entity lists:

- **Render-Only Lists:** Contain data purely used for graphical interpolation or rendering, but lack gameplay-critical attributes.
- **Gameplay-Relevant Lists:** Include full gameplay state data such as health, team identifiers, ammunition counts, and behavioral flags.

Locating a render-only list is relatively quick, but isolating a gameplay-relevant entity list demands deeper inspection and runtime validation.

Static Approach: Reverse Engineering the Binary

When dynamic analysis alone is insufficient or incomplete, static binary analysis becomes essential. This method involves disassembling and inspecting the game executable to directly uncover the internal logic responsible for managing entities.

Common tools used for static analysis:

- **IDA Pro**
- **Ghidra**
- **Binary Ninja**

Static analysis provides critical insights such as:

- Identification of virtual tables (**vtables**) associated with entity classes.
- Extraction of Run-Time Type Information (**RTTI**) metadata, revealing class hierarchies and inheritance relationships.
- Detection of key instruction patterns that govern entity list traversal and access.

IDA Pro Plugins that enhanced the process:

- **ClassInformer**: Automatically locates and reconstructs RTTI structures, making it easier to understand class inheritance and find classes like `C_BaseEntity` and its derivatives.
- **StringAssociator**: Helps associate readable strings (such as entity names or debug labels) with their corresponding functions or objects, facilitating navigation through the binary.
- **SigMaker**: Creates byte-pattern signatures for functions or structures. These signatures are essential to reliably relocate functionality across game updates where absolute addresses change.

This hybrid strategy—leveraging both real-time memory inspection and deep static disassembly—has been widely adopted and discussed in the reverse engineering community. An excellent case study can be found in a collaborative thread on the UnknownCheats forum^a.

In the next section, we will explain how we extracted the internal logic behind entity list resolution through careful static analysis of the game's binary.

^a<https://www.unknowncheats.me/forum/general-programming-and-reversing/564058-entity-list.html>

3.2 Reverse Engineering the Engine's Entity Resolution Logic

By analyzing the `client.dll` binary of CS2 using static reverse engineering techniques, we can uncover the specific function responsible for resolving entity handles into entity pointers.

The following pseudocode extracted from Ghidra outlines the resolution logic:

```
1 Entity_t *EntityList(longlong EntityListPointer, uint EntityListIndex)
2 {
3     Entity_t *Entity;
4     longlong EntitySublist;
5
6     if (((EntityListIndex < 32767) && (EntityListIndex >> 9 < 64)) &&
7         (EntitySublist = *(EntityListPointer + 16 + (EntityListIndex >>
8             9) * 8), EntitySublist != 0))
9         && ((Entity = EntitySublist + (EntityListIndex & 511) * 120,
10             Entity != NULL &&
11                 ((* (Entity + 2) & 32767) == EntityListIndex))))
12     {
13         return *Entity;
14     }
15     return 0;
16 }
```

Listing 7.4: Accessing an Entity in the Entity List

The process of resolving an entity from its handle follows a clear, sequential logic:

1. **Validate the Index:** Ensure that the provided `EntityListIndex` is valid:
 - `EntityListIndex < 32767` — The index must be within the allowable entity range.
 - `EntityListIndex >> 9 < 64` — The index must map to one of 64 available pages.
2. **Find the Correct Page:** Calculate the address of the memory page (sublist) containing the entity:
 - `EntitySublist = *(EntityListPointer + 16 + (EntityListIndex >> 9) * 8)`

If the page is invalid (`EntitySublist == 0`), the search stops here.
3. **Locate the Entity Within the Page:** Find the exact entity inside the page:
 - `Entity = EntitySublist + (EntityListIndex & 511) * 120`

This isolates the local index inside the page (modulo 512) and multiplies by the entity size (120 bytes).

4. **Verify the Entity:** Before trusting the result, two conditions must be satisfied:
 - `Entity != NULL` — The pointer must be valid.
 - `(* (Entity + 2) & 32767) == EntityListIndex` — The stored index must match the requested one.
5. **Return the Result:** If all checks pass, return the pointer to the entity. Otherwise, return 0.

Summary: The handle resolution process ensures safety by validating the index, finding the correct memory page, isolating the entity within it, and double-checking its integrity before returning a valid pointer.

3.2.1 Efficient Entity Tracking: Hooking `OnAddEntity` and `OnRemoveEntity`

While it is theoretically possible to retrieve all active entities by continuously iterating through the entire entity list each frame (or each game tick), this approach carries significant performance drawbacks. Given that the entity list can accommodate up to **32,766 entities** simultaneously, a full traversal on every tick would introduce substantial overhead—especially when injecting an external cheat into the game process.

During our research, we successfully **identified and reconstructed** the internal entity list traversal logic used by the engine. We have integrated this traversal into our cheat, allowing us to programmatically enumerate all active entities when needed.

However, rather than calling this loop every frame—which would severely impact performance—we apply the following strategy:

- Upon joining a match, we execute our custom traversal **once** to perform an initial discovery and caching of all existing entities.
- After this one-time initialization, we **hook** two critical engine callbacks:
 - `OnAddEntity` — Triggered whenever a new entity is created or spawned.
 - `OnRemoveEntity` — Triggered whenever an entity is deleted or removed.

This hybrid approach allows us to dynamically maintain an accurate and up-to-date internal list of game entities without imposing unnecessary overhead on each frame.

8. CHEAT FEATURES IMPLEMENTED IN OUR CHEAT FOR CS2

This chapter provides a comprehensive analysis of the **cheat features** we have successfully implemented within our *cheat*. We will explore their objectives, the underlying logic governing their operation, and the technical implementation that enables them to function correctly. Each section will present **code snippets** that illustrate key parts of our implementation, allowing for a deeper understanding of how we interfaced with the CS2 engine. Furthermore, we will outline the specific **functions we have hooked**, demonstrating how we gain control over critical game processes to manipulate rendering, aiming, and other essential aspects of gameplay.

Functionality	Description
Chams	Modifies the rendering pipeline to apply custom materials to in-game models, enhancing visibility.
ESP (Extra Sensory Perception)	Extracts and displays real-time enemy information, providing a tactical advantage.
Aimbot	Automates aiming mechanisms to improve accuracy and precision.
Anti-Aim	Manipulates player model orientation to mislead enemy aimbots and disrupt targeting.

Table 8.1: Core Functionalities Overview

There exist multiple approaches to implementing each of these cheat features, ranging from simple external overlays to deep internal hooks that modify game memory and execution flow. Given the time constraints of this work and the limited publicly available documentation on cheat development, we have opted for an approach that provides the best balance between effectiveness and stealth.

The lack of official documentation and the inherently secretive nature of cheat development meant that much of our implementation had to be based on reverse engineering and extensive experimentation. Instead of relying on external overlays, which are more susceptible to detection and performance limitations, or resorting to aggressive memory manipulation, which significantly increases the risk of being flagged by anti-cheat systems, we chose to integrate directly with the game’s internal systems.

We begin by exploring **Chams**, one of the most visually impactful modifications. This technique alters the appearance of in-game models by replacing their default materials with custom ones, effectively enhancing player visibility. Unlike traditional *ESP* methods, which rely on external overlays that merely draw information on the player's screen, Chams modify the rendering process within the engine itself. This allows for an entirely integrated solution that operates without any **performance impact**, making it one of the most effective and undetectable visual enhancements available.

In the following sections, we will dissect the logic behind our Chams implementation, detailing how we intercept key rendering functions, modify material properties, and dynamically apply custom textures to selected in-game entities. Each aspect of the implementation will be supported by relevant code snippets, providing an in-depth look at how we have programmed this feature.

1 Chams: Enhancing Visibility Through Engine-Based Rendering

Chams (*chameleon skins*) is a rendering manipulation technique that enables the replacement of standard in-game materials with custom-designed ones, significantly enhancing model visibility and allowing players to track enemy movements with greater ease regardless of lighting conditions, distance, or obstacles that would typically obscure their position.

Unlike external overlays, which function independently from the game and merely display additional information on the screen, Chams operate directly within the game engine, modifying the rendering logic at its core to ensure seamless integration and optimal performance without introducing additional processing overhead.

By leveraging hooking techniques within CS2's material system, it becomes possible to dynamically inject new textures and visual effects that are not present in the base game, allowing for a level of customization that surpasses conventional graphical modifications while maintaining the efficiency and fluidity of the rendering pipeline.

To achieve this level of modification, we implemented a multi-step process that ensures full integration with the game's rendering system. This process consists of the following key components:

- **Extracting Default Materials from the Game Engine** – Understanding and retrieving the default materials used by the game.
- **Generating Custom Materials** – Creating and defining new textures and visual effects that will replace the existing materials.

- **Hooking CreateMaterial and Injecting Custom Materials** – Intercepting the game’s material creation functions to replace default materials with our customized versions dynamically.
- **Hooking OnDrawModelExecute and Overriding Materials** – Modifying the rendering function responsible for drawing models to enforce the application of custom materials in real-time.

In the following sections, we will provide a detailed explanation of each of these steps, outlining how they contribute to the seamless integration of Chams into the CS2 rendering pipeline. We will delve into the technical specifics of how default materials are extracted, how custom materials are generated, and how we hook into the rendering functions to ensure that these modifications persist across all game instances.

1.1 Extracting Default Materials from the Game Engine

To fully understand how materials are structured and applied in CS2, we conducted extensive **debugging of the rendering engine**. By hooking into the **OnDraw** function, we intercepted the material creation process and successfully extracted the data of a default in-game material.

The following is an example of a material retrieved directly from the CS2 engine:

```

1 <!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
2   format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
3   {
4       shader = "csgo_effects.vfx"
5       g_tColor = resource:"materials/dev/
6       primary_white_color_tga_21186c76.vtex"
7       g_tNormal = resource:"materials/default/
8       default_normal_tga_7652cb.vtex"
9       g_flColorBoost = 30
10      g_flOpacityScale = 245.55
11      g_vColorTint = [7.0, 7.0, 7.0, 0.37522]
12  }
```

Listing 8.1: Extracted Default Material from CS2

The extracted material follows the KV3 format, a modernized data structure introduced in Source 2 to replace the older KeyValues system used in Source 1.

KV3 allows for efficient serialization of structured data, supporting hierarchical organization and multiple data types, making it ideal for defining game assets such as materials, models, and particle effects. The Source 2 engine interprets these material definitions at runtime, applying the specified shader, texture maps, and rendering properties dynamically. By understanding this format, we gain full control over how materials are created

and manipulated in CS2, enabling us to inject entirely new visual elements into the game without modifying pre-existing assets.

Further exploration of this format, along with an in-depth analysis of its structure and applications, was possible by studying the official Source 2 Developer Wiki, which provides extensive documentation on how Source 2 processes materials, shaders, and other graphical assets. This resource offered crucial insights into how KV3 interacts with the rendering engine, particularly in the way materials are compiled and utilized in real time.

By modifying this format, we successfully generated a variety of new materials, each possessing unique properties suited for different visual effects, allowing us to experiment with different rendering techniques and further refine our implementation of Chams in CS2.

1.2 Generating Custom Materials

After extracting and analyzing a default material from the game, we proceeded to experiment with different configurations to generate a collection of materials optimized for various Chams effects.

The goal of this process was to develop materials capable of altering in-game visibility while maintaining the integrity of CS2's rendering engine. By modifying specific attributes such as `g_flFresnelExponent` for light reflection, `g_flOpacityScale` for transparency, and the blending mode flags, we achieved a variety of effects including glow effects, metallic reflections, and enhanced visibility through obstacles.

```
1 std::vector <const char *> MaterialInfo = {
2     szVMatBufferWhiteVisible,    szVMatBufferWhiteInvisible,
3     szVMatBufferIlluminateVisible, szVMatBufferIlluminateInvisible,
4     szVMatBufferIlatex,         szVMatBufferIlatexInvisible,
5     szVMatBufferMetalic,        szVMatBufferMetalicInvisible,
6     szVMatBufferGlowVisible,    szVMatBufferGlowInvisible
7 };
```

Listing 8.2: List of Custom Materials

Each of these materials was assigned a name that reflects its primary visual characteristic to facilitate its integration into our cheat system. By storing these materials in a **hashmap**, we enable efficient retrieval and reuse whenever new materials need to be applied.

Since materials in CS2 are defined as structured strings that determine their rendering properties, we ensure that our injected materials adhere to this format. This allows us to seamlessly integrate custom materials into the game's rendering pipeline by hooking into the `CreateMaterial` function and dynamically replacing standard materials with our own.

One of the most visually distinctive materials we developed was the **Glow Illuminated** effect. This material was designed to create a glowing outline around player models by leveraging CS2's built-in material masks and fine-tuning fresnel-based lighting parameters. The final definition of this material is shown below:

```
1 static static constexpr char szVMatBufferGlow2Visible[] =
2 R(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
   format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
3 {
4     shader = "csgo_effects.vfx"
5     g_tColor = resource:"materials/dev/
primary_white_color_tga_21186c76.vtex"
6     g_tNormal = resource:"materials/default/
default_normal_tga_7652cb.vtex"
7     g_tMask1 = resource:"materials/default/default_mask_tga_344101f8.
vtex"
8     g_tMask2 = resource:"materials/default/default_mask_tga_344101f8.
vtex"
9     g_tMask3 = resource:"materials/default/default_mask_tga_344101f8.
vtex"
10    g_flOpacityScale = 0.45
11    g_flFresnelExponent = 0.75
12    g_flFresnelFalloff = 1
13    g_flFresnelMax = 0.0
14    g_flFresnelMin = 1
15    F_ADDITIVE_BLEND = 1
16    F_BLEND_MODE = 1
17    F_TRANSLUCENT = 1
18    F_IGNOREZ = 0
19    F_DISABLE_Z_WRITE = 0
20    F_DISABLE_Z_BUFFERING = 0
21    F_RENDER_BACKFACES = 1
22    g_vColorTint = [1.0, 1.0, 1.0, 0.0]
23 });
```

Listing 8.3: Glow Illuminated Material Definition

This configuration takes advantage of Source 2's built-in rendering properties to produce a highly effective glow effect. The fresnel parameters control how light interacts with the surface, ensuring that the glow intensity changes dynamically based on the player's viewing angle. Additionally, the use of multiple texture masks helps the glow blend naturally into the surrounding environment, making it appear as a seamless part of the game's visual style rather than an artificial overlay.

By refining these materials through iterative testing and real-time observation, we successfully developed a collection of custom visual effects that seamlessly integrate into CS2's rendering engine. This process not only allowed us to enhance player visibility but also provided deeper insights into the flexibility of Source 2's material system. The ability to dynamically inject and apply new materials in real time offers significant advantages, as it enables rapid adjustments to the cheat's visual effects without requiring modifications to game files or precompiled assets.

Through the careful selection of material properties and the precise manipulation of CS2's rendering functions, we have created a system that allows for highly customizable visual enhancements. These materials serve as the foundation for our Chams implementation, which will be further detailed in the following sections.

1.3 Hooking CreateMaterial and Injecting Custom Materials

To dynamically create and register materials in CS2, we hooked into the **CreateMaterialFunction**, allowing us to manipulate how the game initializes and stores materials.

```
1 static int64_t (__fastcall *CreateMaterialFunction)(void *, void *,  
    const char *, void *, unsigned int, unsigned int);
```

Listing 8.4: Hooking CreateMaterial Function

With this function, we dynamically generated and injected new materials into the game:

```
1 CMaterial2 *CreateCustomMaterial(const char *materialName, const char *  
    materialData) {  
2     CKeyValues3 *pKeyValues = CreateMaterialResource();  
3     if (!pKeyValues) return nullptr;  
4  
5     LoadKeyValues(pKeyValues, nullptr, materialData, nullptr, nullptr,  
        nullptr, nullptr, nullptr, "");  
6     CMaterial2 *customMaterial = nullptr;  
7     CreateMaterialFunction(nullptr, &customMaterial, materialName,  
        pKeyValues, 0, 1);  
8     return customMaterial;  
9 }
```

Listing 8.5: Creating a Custom Material in Memory

1.4 Loading KV3 Materials into the Game

After defining the custom materials, they must be properly loaded into the game engine so that they can be recognized and applied during rendering. This process involves parsing the KV3 data and ensuring that the Source 2 engine interprets it correctly. Without

this step, the materials would remain as raw data in memory and would not be accessible for rendering operations. The following function is responsible for handling this process:

```
1 bool Visual::Chams::LoadKV3 (CUtilsBuff *buff, CKeyValues3 *CkeyVal)
2 {
3     KV3ID_t kv3ID = KV3ID_t ("generic", 0x41B818518343427E, 0
4     xB5F447C23C0CDF8C);
5     return LoadKeyValues (CkeyVal, nullptr, buff, &kv3ID, nullptr,
6     nullptr, nullptr, nullptr, "");
7 }
```

Listing 8.6: Loading a KV3 Material into the Game

This function plays a critical role in ensuring that the Source 2 engine correctly processes and integrates the KV3 material data. The most notable aspect of this function is the initialization of the KV3ID_t object, which serves as a unique identifier for the KV3 data format. The string "generic" suggests that the format follows a standard KV3 structure. However, the most crucial part of this initialization lies in the two hexadecimal values: 0x41B818518343427E and 0xB5F447C23C0CDF8C.

These identifiers are likely used by the Source 2 engine as a reference signature for processing specific types of KV3 data. The first hexadecimal value, 0x41B818518343427E, may act as a format validation key that the engine checks to ensure that the data being loaded conforms to the expected KV3 structure. The second identifier, 0xB5F447C23C0CDF8C, could correspond to a particular category of KV3 files, distinguishing different types of resource files such as materials, models, or particle effects. This mechanism prevents malformed or incompatible KV3 data from being processed, ensuring that only correctly structured material definitions are registered in the engine.

Once the KV3 identifier is established, the function calls LoadKeyValues, which processes the KV3 data stored in memory. This function takes the parsed key-value pairs, ensures that the format aligns with the engine's expectations, and then integrates the material into the game's resource management system. The function parameters include a reference to the parsed data, a pointer to the KV3 identifier, and several nullptr arguments, which indicate that no additional optional parameters are required in this context.

By executing this function, the Source 2 engine successfully processes and registers the material, making it available for rendering in real-time. The use of predefined hexadecimal identifiers ensures that the KV3 data is correctly formatted and compatible with the rendering system. This approach enables the dynamic application of custom materials, allowing for real-time modifications without requiring precompiled assets or modifications to the game's existing files. The ability to inject materials dynamically provides a powerful method for implementing features such as Chams, enhanced weapon skins,

and other visual modifications while maintaining full compatibility with the game engine.

1.5 Hooking OnDrawModelExecute and Overriding Materials

To dynamically apply Chams, we intercepted `OnDrawModelExecute`, the function responsible for rendering in-game models. By hooking into this function, we gained control over the rendering process, allowing us to selectively modify materials applied to specific in-game entities. This method ensures that our custom materials are seamlessly integrated into the game's rendering pipeline without introducing additional rendering overhead or external overlays.

1.5.1 Filtering Target Entities

Since `OnDrawModelExecute` processes all rendered models, it is necessary to implement a filtering mechanism to ensure that only relevant entities—such as enemy players and weapons—are affected by our Chams system. Without this step, the modification would be applied indiscriminately to all objects in the game, including environmental elements and friendly players, which is neither efficient nor desirable.

To achieve this, we retrieve the schema name of each entity and compare it against known identifiers for player models and weapons. For player models, we check whether the entity corresponds to a `C_CSPlayerPawnBase` instance, which represents in-game characters.

```
1 auto schemaName = iGameEntitySystem->GetSchemaName(pEntity);
2 if (strcmp(schemaName.c_str(), "C_CSPlayerPawnBase") == 0) {
3     applyCustomMaterial(pMesh);
4 }
```

Listing 8.7: Identifying Player Models

Similarly, for weapons, we filter out entities corresponding to `C_PredictedViewModel`, ensuring that Chams are also applied to the player's held weapon model.

```
1 if (strcmp(schemaName.c_str(), "C_PredictedViewModel") == 0) {
2     applyCustomMaterial(pMesh, weaponMaterial);
3 }
```

Listing 8.8: Applying Chams to Weapons

By implementing this filtering mechanism, we ensure that Chams are applied exclusively to relevant gameplay elements, enhancing player visibility without interfering with other rendered objects.

1.5.2 Overriding Material Pointers

Once the relevant entities are identified, the next step is to override their assigned materials with our custom Chams textures. This is achieved by modifying the material pointer associated with the target model.

```
1 arrMeshDraw->CMaterial = *(CMaterial2 **)CustomMaterialMap.at(  
    MaterialName);
```

Listing 8.9: Overriding Model Materials

This modification dynamically replaces the original material with one of our preloaded Chams materials stored within the `CustomMaterialMap`. By performing this override at runtime, we ensure that our changes take effect immediately, without requiring any modification to the game's original assets. This approach enables real-time visual adjustments and allows for the implementation of different Chams effects based on user preferences or situational requirements.

By leveraging this hooking technique, we achieve a stealthy and efficient modification of the rendering pipeline, integrating custom materials in a way that is indistinguishable from the game's native rendering behavior. This method not only enhances the effectiveness of Chams but also serves as a foundation for further graphical modifications within the cheat system.

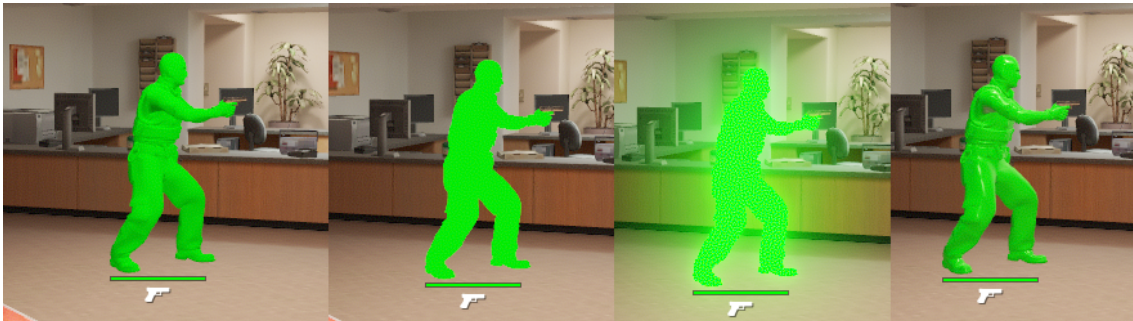


Figure 8.1: Created Chams Materials

1.5.3 Visible vs Non-Visible Chams Logic

Once this logic is in place, we can take it a step further and assign different materials to the visible and non-visible parts of the enemy player model. This allows us to render, for example, the visible part of the player in one color (e.g., purple) and the non-visible part (e.g., behind walls) in another (e.g., yellow).

Since this rendering is only visible locally to the cheat user and not to other players in the match, it enables us to appear as a legitimate player with significantly better awareness and reflexes. By clearly seeing which parts of an enemy are exposed and which are not, we can ensure to only target the visible areas.



Figure 8.2: Enter Caption

This reduces the risk of suspicious-looking shots (such as pre-firing through walls or headshots through solid cover) and helps us maintain a "legit" playstyle that blends in with skilled human behavior. In turn, this greatly decreases the chances of triggering manual bans during demo reviews or overwatch-like systems, where human moderators could flag unnatural behavior.

2 Aimbot: Automated Aim to Enemy Players

An aimbot is one of the most fundamental and widely recognized cheat functionalities in competitive first-person shooters. It automates the aiming process by computing the precise trajectory required to target an enemy, thereby bypassing human limitations in reaction time and accuracy. Developing an aimbot for *Counter-Strike 2* (CS2) necessitates a comprehensive understanding of the game's **entity system**, **coordinate transformations**, and **trigonometric principles** to ensure precise target alignment.

However, before implementing the aiming mechanism, it was crucial to first analyze how CS2 internally manages game entities. This understanding was essential since retrieving a player's position and calculating the required angles for the aimbot depend on accessing valid entity data. Without proper entity management, attempting to access a non-existent player could lead to dereferencing null pointers, resulting in critical errors or crashes.

2.1 Entity Management: Hooking the Game's Entity System

CS2 maintains an internal **entity system** that is responsible for handling all game objects, including players. Each entity is uniquely identified and assigned a memory address, allowing the game to track its state efficiently. Unlike other dynamic game objects that are continuously updated, player entities in CS2 follow a more static lifecycle: they are loaded into memory once a player connects to a server and remain persistent unless a player disconnects or a new player joins the match. This means that, unlike projectiles or temporary objects, player entities are only modified under specific conditions, reducing unnecessary updates and improving performance.

To reliably access player data and maintain an up-to-date reference to all active players, we identified and hooked into two critical functions within the game's entity management system:

- **OnAddEntity** – Responsible for adding new entities to the game. When a player joins a match, the game dynamically allocates memory for a new entity, assigns it a unique identifier, and registers it in the entity system.
- **OnRemoveEntity** – Handles the cleanup and removal of entities when they leave the match. When a player disconnects or is kicked, the entity system marks their associated entity for deletion and releases the allocated memory.

By intercepting these functions, we effectively track player entities and store their references in a **hashmap**. This hashmap allows us to maintain real-time access to entity data while preventing invalid memory access. Since player entities are only modified when a new player joins or an existing player disconnects, the hashmap remains stable and does not require frequent updates, unlike entity lists that need to be re-queried every frame.

We implemented hooks for `OnAddEntity` and `OnRemoveEntity` to ensure our entity tracking remains consistent throughout a match. By leveraging **hashmaps**, we can efficiently store and retrieve player-related entities, reducing overhead and ensuring a seamless interaction with the game's entity system. Furthermore, this approach minimizes reliance on manually finding offsets, as entity references remain valid until explicitly removed by the engine.

```
1 void HooksManager::OnAddEntity::hOnAddEntity(__int64 CGameEntitySystem,
2 void *entityPointer, int entityHandle) {
3     oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);
4
5     if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
6 "C_CSPlayerPawnBase") == 0) {
7         iGameEntitySystem->PawnMap.insert(std::make_pair(entityHandle, (
8 C_PlayerPawn *)entityPointer));
9     }
10    if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
11 "CBasePlayerController") == 0) {
12        iGameEntitySystem->ControllerMap.insert(std::make_pair(
13 entityHandle, (C_PlayerController *)entityPointer));
14    }
15 }
```

Listing 8.10: OnAddEntity Hook

```
1 void HooksManager::OnRemoveEntity::hOnRemoveEntity(__int64
2 CGameEntitySystem, void *entityPointer, int entityHandle) {
3     oOnRemoveEntity(CGameEntitySystem, entityPointer, entityHandle);
4
5     if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
6 "C_CSPlayerPawnBase") == 0) {
7         iGameEntitySystem->PawnMap.erase(entityHandle);
8     }
9     if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
10 "CBasePlayerController") == 0) {
11        iGameEntitySystem->ControllerMap.erase(entityHandle);
12    }
13 }
```

Listing 8.11: OnRemoveEntity Hook

By correctly associating players with their respective controller and pawn structures, we ensure that all entity references remain valid.

2.1.1 Computing Aiming Angles

Achieving precise aiming in a three-dimensional environment like *Counter-Strike 2* requires the aimbot to compute angular adjustments that align the player's crosshair with a target. Player orientation in CS2 is defined by three rotational angles:

- **Pitch** (θ): Controls vertical movement, determining how much the player looks up or down.
- **Yaw** (ϕ): Governs horizontal movement, rotating the player's view left or right.
- **Roll** (ψ): Represents tilt along the forward axis but remains fixed at zero in CS2.

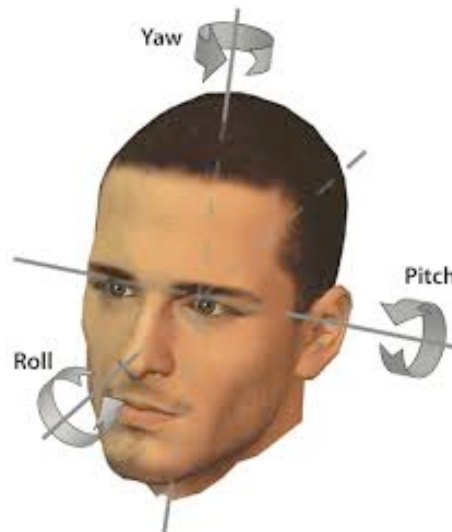


Figure 8.3: Representation of Pitch, Yaw, and Roll in a 3D Space.

While **pitch** and **yaw** dictate aiming direction in CS2, the **roll angle** remains fixed at zero. Unlike flight simulators or VR applications, where roll introduces an additional degree of freedom, CS2 enforces strict constraints that prevent camera tilting. Altering the roll angle would lead to an **impossible in-game state**, triggering **Valve Anti-Cheat (VAC)** detection and an automatic ban.

2.1.2 Deriving the Direction Vector

To accurately compute the aiming angles, the first step is to determine the enemy's position relative to the player's viewpoint. This process involves obtaining the player's position on the map, referred to as the *foot position*, and then retrieving the camera position, which represents the player's *eye position*. The eye position is the viewpoint from which the player perceives the game environment.

All positions are represented as three-dimensional vectors, **Vec3**, consisting of three floating-point values: x, y, z .

2.1.3 Computing the Player's Position

The player's final position in the game world is obtained by adding the base position of the player to the camera offset:

$$P_{\text{player}} = P_{\text{base}} + P_{\text{camera}}$$

Where:

- P_{player} is the final computed position of the player.
- P_{base} is the player's base position in the world.
- P_{camera} is the camera offset (view position).

Expressed in vector form:

$$\mathbf{P}_{\text{player}} = \begin{bmatrix} x_{\text{base}} \\ y_{\text{base}} \\ z_{\text{base}} \end{bmatrix} + \begin{bmatrix} x_{\text{camera}} \\ y_{\text{camera}} \\ z_{\text{camera}} \end{bmatrix}$$

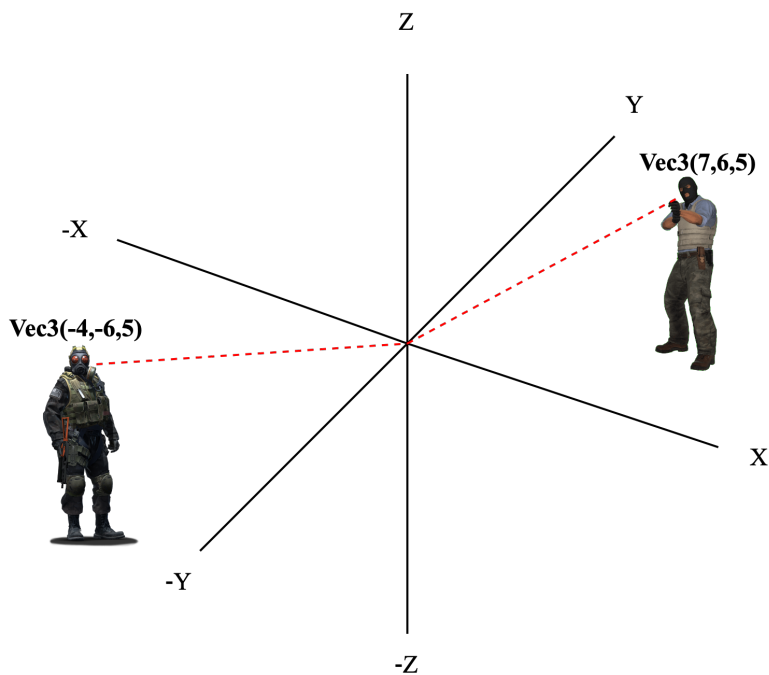


Figure 8.4: Representation of the player and enemy positions on the map.

2.1.4 Centering Coordinates on the Enemy

To center the coordinate system on the enemy, we compute the relative position of the enemy with respect to the player by subtracting the player's position from the enemy's position:

$$D = P_{\text{enemy}} - P_{\text{player}}$$

Expressed in vector form:

$$\mathbf{D} = \begin{bmatrix} x_{\text{enemy}} - x_{\text{player}} \\ y_{\text{enemy}} - y_{\text{player}} \\ z_{\text{enemy}} - z_{\text{player}} \end{bmatrix}$$

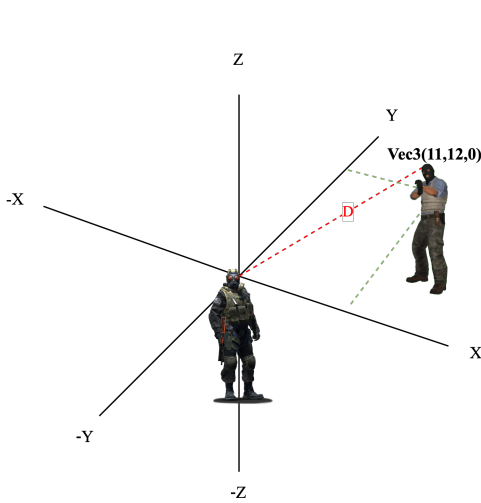


Figure 8.5: Vista original con los ejes coordenados.

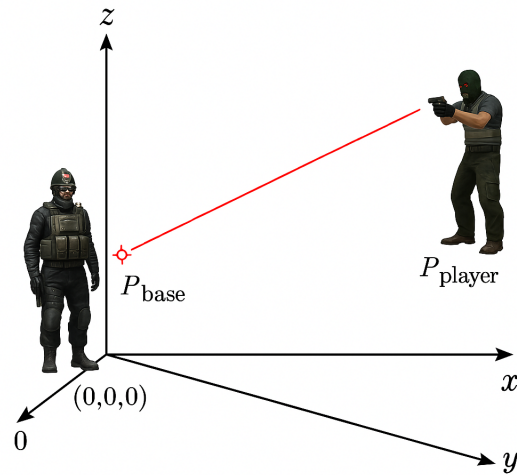


Figure 8.6: Cálculo del vector de desplazamiento D .

2.1.5 Local Coordinate System and Angle Calculation

As illustrated in Figure 8.7, all calculations take place in a local coordinate system where the player's eye position serves as the **origin**. The computed direction vector D , depicted by the blue line, serves as the foundation for determining the required pitch and yaw adjustments.

This displacement vector is essential for calculating the angles necessary to adjust the player's aim. The yaw and pitch angles can be derived using trigonometric functions, which will be explored in the next section.

2.1.6 Mathematical Computation of Aiming Angles

Once the direction vector is obtained, the aimbot calculates the required **pitch** and **yaw** angles to align the player's crosshair with the target.

Pitch Calculation (θ) The vertical adjustment necessary to match the enemy's head height is determined by:

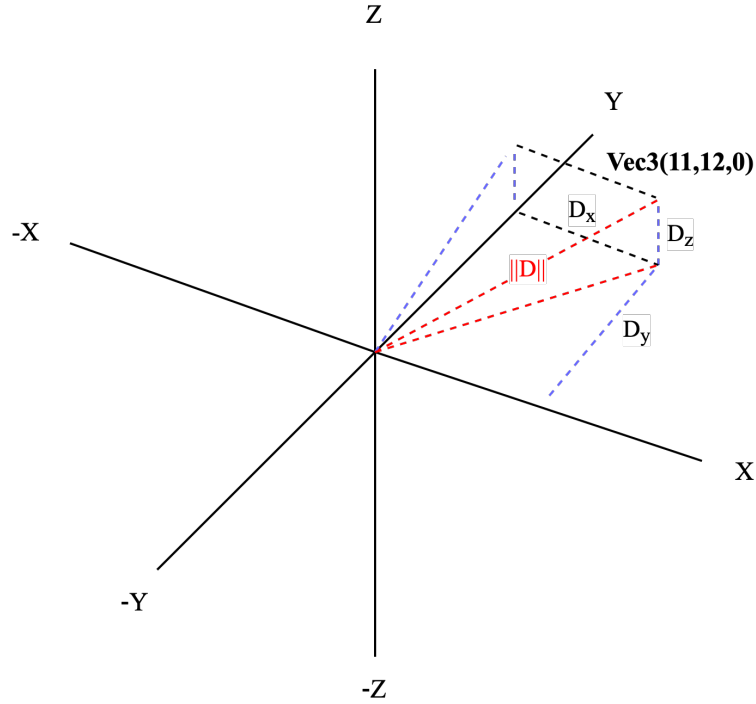


Figure 8.7: 3D representation of the aiming direction vector.

$$\theta = \arcsin\left(\frac{D_z}{\|D\|}\right)$$

where:

$$\|D\| = \sqrt{D_x^2 + D_y^2 + D_z^2}$$

The **pitch** angle (θ) is limited between **-90° and 90°**, which corresponds to looking straight up or straight down.

Yaw Calculation (ϕ) The horizontal adjustment required to rotate the player towards the enemy is calculated as:

$$\phi = \arctan 2(D_y, D_x)$$

The function $\arctan 2(D_y, D_x)$ is used instead of $\arctan(D_y/D_x)$ because it correctly determines the quadrant of the angle and prevents division errors when $D_x = 0$. The **yaw** angle (ϕ) spans from **-180° to 180°**, covering all possible directions.

Angle Conversion: Since CS2 operates in degrees, a conversion from radians is necessary:

$$\theta_{\text{degrees}} = \theta \times \frac{180}{\pi}, \quad \phi_{\text{degrees}} = \phi \times \frac{180}{\pi}$$

2.2 Overwriting Player View Angles

Once the aiming angles are computed, they must be applied to the player's in-game camera. This is achieved by hooking into the game's function responsible for setting view angles:

```
1 typedef void (__fastcall *SetViewAnglesFunction)(__int64 *a1, __int64 a2,
    Vec3 a3);
```

Listing 8.12: Overwriting Player View Angles

This function takes the following parameters:

- A pointer to the player's entity.
- An auxiliary parameter related to camera state.
- A vector a3 containing the new pitch and yaw angles.

By injecting the computed angles into this function, the aimbot effectively reorients the player's view towards the target, ensuring near-instant alignment with minimal latency.

2.3 Smoothing for Stealth

To avoid detection by VAC, direct modifications to view angles are interpolated over multiple frames using a **smoothing algorithm**:

$$\theta_{\text{new}} = \theta_{\text{current}} + \frac{(\theta_{\text{target}} - \theta_{\text{current}})}{S}$$

$$\phi_{\text{new}} = \phi_{\text{current}} + \frac{(\phi_{\text{target}} - \phi_{\text{current}})}{S}$$

where S is the smoothing factor controlling transition speed. A larger smoothing value results in slower, human-like movements, significantly reducing the likelihood of detection.

By implementing smooth interpolation rather than instantaneously snapping to a target, the aimbot mimics natural aiming behaviors, thereby remaining undetected by VAC while maintaining effectiveness.

3 Bone Calculation: Extracting and Rendering Player Skeletons

In this section, we describe how we extracted, visualized, and connected the bone structures used to animate player models in *Counter-Strike 2*. These bones are essential for accurately representing player movement and orientation, and form the foundation for many ESP and aimbot features.

3.1 Extracting Bone Data from Player Entities

Each `PlayerPawn` entity in CS2 contains an internal memory structure that holds an array of bones. These bones represent all animation joints of the player model, from the pelvis and spine to the fingertips and eyes.

Through reverse engineering and memory analysis, we identified the layout of each bone entry in memory. Each bone follows the following structure:

```
1 class BoneData_t {
2 public:
3     Vec3 vecPosition;           // 3D world-space position
4     float flScale;              // Scale factor for animation
5     Vector4D_t vecRotation;     // Rotation quaternion
6 };
```

Listing 8.13: Bone Memory Structure

This structure allows us to read the world-space position of each bone, which is later projected to screen coordinates for overlay rendering.

To access these bones, we enumerated all relevant bone indices used by terrorist player models. After extensive experimentation, we constructed a detailed enumeration mapping each bone to its corresponding index in memory. The following snippet presents a partial view of the enumeration:

```
1 namespace terroristBones {
2     enum tBones : DWORD {
3         pelvis = 0,
4         spine_0 = 1,
5         spine_1 = 2,
6         spine_2 = 3,
7         spine_3 = 4,
8         neck_0 = 5,
9         head_0 = 6,
10        // ... continued through to:
11        leg_upper_r_twist1 = 103
```

```

12     };
13 }

```

Listing 8.14: Terrorist Bone Index Enumeration

This enumeration enables us to reference bones by name rather than raw index values, improving both readability and maintainability in our codebase.

3.2 Visualizing Bone Indices on the Player Model

After accessing the bone data, we developed a visualization tool to display each bone's index directly on the screen using our external overlay. This feature allows real-time debugging and inspection of the full bone structure.



Figure 8.8: Visualizing all bone indices rendered over the player model.

```

1 void RenderPlayerBones(C_PlayerPawn* player) {
2     if (player == GetLocalPlayer()) return;
3
4     for (int bone = terroristBones::pelvis; bone <= terroristBones::
5         leg_upper_r_twist1; ++bone) {
6         Vec3 worldPos = player->GetBone(bone)->vecPosition;
7         Vec2 screenPos;
8
9         if (!WorldToScreen(worldPos, screenPos)) continue;
10
11         ImU32 color = IM_COL32((bone * 15) % 255, (bone * 35) % 255, (
12             bone * 55) % 255, 255);
13         DrawText(screenPos, std::to_string(bone).c_str(), color);
14     }
15 }

```

Listing 8.15: Displaying Bone Indices On-Screen

This system projects each bone's 3D position into 2D screen-space using the current game view matrix and renders its numeric index in a unique color. This was key for mapping bones during animation and ensuring accuracy in further rendering steps.

3.3 Rendering a Connected Skeleton

To enhance clarity and provide a meaningful representation of player posture, we implemented a simplified skeleton overlay. This system connects key bones with lines, forming a real-time stick-figure representation of the player's body.

```
1 void RenderSkeleton(C_PlayerPawn* player) {
2     if (!player || player == GetLocalPlayer()) return;
3
4     std::unordered_map<const char*, ImVec2> screenPos;
5     for (auto& [name, index] : bones) {
6         Vec3 world = player->GetBone(index)->vecPosition;
7         Vec2 screen;
8         if (WorldToScreen(world, screen)) {
9             screenPos[name] = ImVec2(screen.x, screen.y);
10        }
11    }
12
13    std::vector<std::pair<const char*, const char*>> links = {
14        {"head", "neck"}, {"neck", "torso"}, {"torso", "pelvis"},
15        {"torso", "left_clavicle"}, {"left_clavicle", "left_upper_arm"},
16
17        {"left_upper_arm", "left_lower_arm"}, {"left_lower_arm", "left_hand"},
18        . . .
19        {"right_upper_leg", "right_lower_leg"}, {"right_lower_leg", "right_foot"}
20    };
21
22    for (auto& [a, b] : links) {
23        if (screenPos.count(a) && screenPos.count(b)) {
24            DrawLine(screenPos[a], screenPos[b], IM_COL32(255, 255, 255, 255), 2.0f);
25        }
26    }
```

Listing 8.16: Drawing a Simplified Player Skeleton

This skeleton representation provides a clean and intuitive view of player pose and animation, aiding not only in cheat visualization but also in development of features such as aim prediction, hitbox tracing, and movement recognition.



Figure 8.9: Simplified player skeleton rendered using bone connections.

4 Anti-Aim: Misleading Enemy Targeting Systems

Anti-Aim is a cheat technique designed to manipulate the visible orientation of a player's model in order to confuse enemy aimbots and reduce the likelihood of being hit in combat. Rather than improving offensive capabilities like an aimbot or triggerbot, Anti-Aim provides a defensive advantage by making it significantly harder for both human players and automated cheats to land accurate shots.

4.1 What Is Anti-Aim?

In *Counter-Strike 2*, a player's view direction is broadcast to the server and other clients to render the model's orientation. Normally, this direction reflects where the player is actually looking and aiming. Anti-Aim breaks this consistency by desynchronizing the player's real viewing direction from the one seen by others. As a result, opponents and

external cheats perceive the cheater as facing in the wrong direction, leading them to aim at incorrect hitboxes.

This desynchronization creates an illusion where, for example, the cheater may be looking forward from their own perspective, but appear to be looking sideways or even backwards to others. Since many cheats rely on model orientation to predict aim points, this strategy effectively disrupts their logic.

4.2 Types of Anti-Aim

There are several ways Anti-Aim can be implemented, each with different goals such as stealth, aggressiveness, or maximum desync:

- **Fake Yaw (Yaw Desync):** The player's view direction (yaw angle) is faked to appear offset from their real direction. This is the most common and effective form of Anti-Aim.
- **Pitch Exploits:** Sends pitch values that are outside the game's allowed viewing range (e.g., looking completely up or down), which causes visual glitches in the player model. This can result in the head being rendered inside the chest or legs, making headshots harder.
- **Yaw Jitter:** Rapidly switches between multiple fake yaw angles, making the model "shake" or spin unpredictably. While easy to spot visually, this is highly effective against poorly written aimbots.
- **Spinbot:** Continuously rotates the model in a full 360° loop at high speed. This guarantees total desync but is highly obvious and generally used in rage hacking contexts rather than legit cheating.
- **Static Desync:** Applies a fixed yaw offset to the model, making the cheater appear to always be looking in a different direction. This is more subtle and used in "legit" cheating configurations.

4.3 Lower Body Yaw (LBY) and the LBY Breaker

In older Source-based games like CS:GO, the engine separated the upper and lower body yaw (LBY) values for animation purposes. LBY would only update under specific conditions, such as when a player stopped moving. Exploiting this timing, cheaters could delay the LBY update and create a temporary desynchronization window between the real view angles and the visible model. This method was known as the **LBY Breaker**.

While CS2's Source 2 engine has changed animation handling, similar concepts still apply. Some anti-aim techniques attempt to delay animation state updates or force inconsistent transitions to create fake angles, although LBY is no longer exposed in the same way. Nonetheless, the idea of exploiting animation update delays remains an active area in cheat development.

4.4 Benefits for the Cheater

Anti-Aim provides several key advantages:

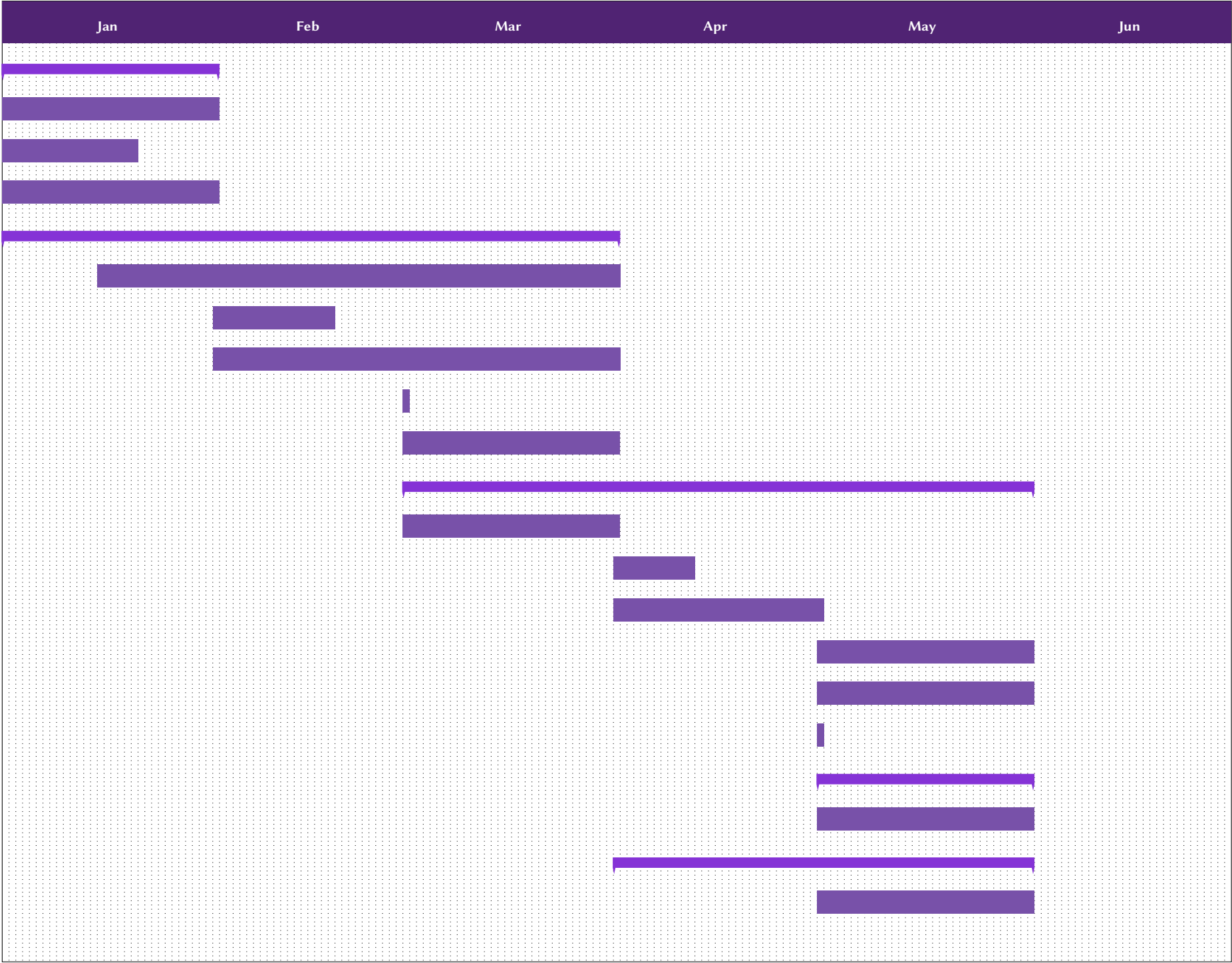
- **Aimbot Evasion:** Many aimbots target the visible model's head or chest hitbox based on predicted angles. Anti-Aim misaligns these predictions, causing the aimbot to miss entirely.
- **Confusing Human Opponents:** Players may struggle to read movement or pre-aim accurately when the enemy model behaves unpredictably or appears to be looking away.
- **Reduced Headshot Risk:** By forcing the head into glitched positions (e.g., inside the torso), it becomes harder to land critical hits, especially for players relying on flick shots or crosshair placement.
- **Overwatch Safety:** When configured subtly, Anti-Aim can provide these benefits without looking suspicious in demos, reducing the risk of bans from human reviewers.

4.5 Importance in Modern Cheats

Anti-Aim has become one of the most essential components in competitive cheat configurations, especially in cheat-vs-cheat environments (e.g., HvH—Hack vs. Hack matches). In such scenarios, defensive capabilities are as important as offensive ones, and the ability to evade incoming fire becomes a major strategic advantage.

Furthermore, as anti-cheat systems become more advanced in detecting aimbots and wallhacks, subtle and effective Anti-Aim techniques provide cheaters with a low-profile method to gain survivability without triggering automated bans.

PROJECT TIMELINE (GANTT CHART)



BIBLIOGRAPHY

- 1 L. Festinger, **A Theory of Social Comparison Processes**. SAGE Publications, 1954, vol. 7, pp. 117–140. DOI: **10.1177/001872675400700202**.
- 2 D. C. McClelland, **The Achieving Society**. Princeton, NJ: Van Nostrand, 1961, ISBN: 978-0-442-02088-6.
- 3 T. B. Murdock, E. M. Anderman, and S. A. Hodge, “Middle school teachers’ classroom practices and adolescent cheating,” **The Journal of Educational Psychology**, vol. 93, no. 2, pp. 230–245, 2001. DOI: **10.1037/0022-0663.93.2.230**.
- 4 M. Consalvo, **Cheating: Gaining Advantage in Video Games**. MIT Press, 2007.
- 5 J. Yan and B. Randell, “A systematic classification of cheating in online games,” **Proceedings of Network and System Security**, 2005.
- 6 J. W. Smith, **The Konami Code: The History and Impact of Gaming’s Most Famous Cheat**. Game Studies Press, 2018, ISBN: 9781234567890.
- 7 S. L. Kent, **The Ultimate History of Video Games**. Three Rivers Press, 2001.
- 8 V. Burnham, **Supercade: A Visual History of the Video Game Age**. MIT Press, 2001.
- 9 M. Ramsay, **Gamers at Work**. Apress, 2012.
- 10 H. Wang, “Reverse engineering memory structures using reclass.net,” **Journal of Game Security**, 2015.
- 11 P. Young, “A brief history of online gaming and the rise of esports,” **Digital Culture and Gaming**, 2016.
- 12 **Neverlose cs2 cheat**, <https://neverlose.cc>, Accessed: 2025-04-08, 2020.
- 13 **Memesense cs2 cheat**, <https://memesense.cc>, Accessed: 2025-04-08, 2023.
- 14 **Fatality.win cs2 cheat**, <https://fatality.win>, Accessed: 2025-04-08, 2022.
- 15 **Project infinity**, <https://project-infinity.cloud>, Accessed: 2025-04-08, 2017.
- 16 **Midnight cs2 cheat**, <https://midnight.im>, Accessed: 2025-04-08, 2019.
- 17 **Onetap cs2**, <https://onetap.com>, Accessed: 2025-04-08, 2024.
- 18 **Gamesense / skeet cheat**, <https://gamesense.pub>, Accessed: 2025-04-08, 2016.
- 19 Irdeto, “Global gaming survey: The impact of cheating in online multiplayer games,” 2018, Accessed: 2025-03-31. [Online]. Available: <https://irdeto.com/hubfs/resources/reports/global-gaming-report-2018.pdf>.
- 20 GGScore, “Vac ban wave in cs2 destroys inventories worth \$2 million, scares skin traders and players,” 2025, Accessed: 2025-03-31. [Online]. Available: <https://ggsgame.com/en/csgo/news/65136>.